



**TECHNISCHE
UNIVERSITÄT
DRESDEN**



ComNets
Deutsche Telekom Chair
of Communication Networks

Faculty of Electronic and Information Institute of Communication Engineering

Chair of Communication Networks

Master Thesis

Randomized Identification Codes with a Focus on K-Identification

Chongyang Zheng

Born on: 10th October 1997 in Xi'an

Course: Nanoelectronic Systems

Matriculation number: 5198083

Matriculation year: 2023

21st January 2026

Supervisors

Juan A. Cabrera

Caspar von Lengerke

Supervising professor

Prof. Dr. h.c. Fitzek



Task for Master Thesis of

Mr. Chongyang Zheng

Academic year: ET/

, matriculation number:

Topic: Randomized Identification Codes with a Focus on K-Identification

Task description:

Identification via channels (ID) asks receivers to decide whether a specific message was sent, rather than to reconstruct it. Randomized ID codes achieve doubly-exponential message sets in blocklength and enable vanishing Type-I (false-accept) and Type-II (miss) error probabilities at positive ID rate. K-Identification (K-ID) generalizes this: the receiver asks whether the transmitted message belongs to a set of K candidates. K-ID is relevant for goal/semantics-driven communication (e.g., digital twins), where the action depends only on whether the current state falls into a small set of actionable hypotheses, not on exact message recovery. This thesis will study randomized ID codes with emphasis on K-ID, and extend the ideas and evaluation from “Stopping the Data Flood: Post-Shannon traffic reduction in digital-twins applications” to the K-ID setting. The work should balance theory, algorithms, and software-driven experiments using our provided library.

The student is expected to:

Survey the state of the art:

- Core ID concepts: randomized encoders/acceptance regions, ID capacity, error exponents, and operational interpretations; contrast with Shannon transmission.
- Define K-ID precisely (membership in a K-set) and position it vs. related notions (multiple-object ID, generalized ID; deterministic vs randomized variants).

Formulate the problem & metrics for K-ID:

- Formal model (channel class, randomization/common randomness budget, decoder tests).
- Report both Type-I/Type-II errors and computational metrics (encode/decode time, seed length, memory), plus finite-blocklength behavior.

Replicate & extend the digital-twins baseline to K-ID:

- Reproduce key experiments/results from the “Stopping the Data Flood” study (baseline ID) with our software stack.
- Design K-ID experiments that reflect digital-twin quantifying traffic reduction vs. reliability/latency.

Use our software library to build a modular K-ID test bench:

- Implement randomized ID/K-ID encoders and decoders (acceptance region tests) as pluggable components.

Design and run a simulation campaign:

- Measure KPIs such as: Type-I/II error curves, empirical exponents, queries per symbol, and compute/memory usage. Quantify traffic reduction vs. reliability/latency

Deliver a comparative assessment with clear takeaways:

- When do systems benefit from K-ID?
- Tradeoffs among randomization, complexity, and reliability.

Supervisors: Juan A. Cabrera, Caspar von Lengerke

Started at: 01.09.2025

To be submitted by: 11.02.2026

Prof. Dr.-Ing. Dr. h.c. Fitzek
Responsible Tutor



Statement of authorship

I hereby certify that I have authored this document entitled *Randomized Identification Codes with a Focus on K-Identification* independently and without undue assistance from third parties. No other than the resources and references indicated in this document have been used. I have marked both literal and accordingly adopted quotations as such. During the preparation of this document I was only supported by the following persons:

Juan A. Cabrera
Caspar von Lengerke

Additional persons were not involved in the intellectual preparation of the present document. I am aware that violations of this declaration may lead to subsequent withdrawal of the academic degree.

Dresden, 21st January 2026

Chongyang Zheng



Abstract

Identification via channels (ID) asks a receiver to decide whether a specific hypothesis about the transmitted message is true, rather than reconstructing the message itself. This document provides precise definitions, key capacity results, error metrics, and an experimental plan tailored to K-Identification (K-ID), where the receiver asks whether the message belongs to a set of K candidates. It contrasts randomized vs. deterministic (DI) variants, formulates problem and metrics, and outlines a modular test bench and simulation campaign aligned with the provided software stack.

Contents

List of Figures	IV
List of Tables	VI
Acronyms	VII
1 Introduction	1
2 Background and Related Work	3
2.1 From Data Reconstruction to Semantic Identification	3
2.2 Alternative Synchronization Strategies	4
2.2.1 Full State Transmission (The Baseline)	4
2.2.2 Delta Encoding and the Drift Problem	4
2.2.3 Event-Triggered Control (ETC)	4
2.2.4 The Identification Advantage	5
2.3 Encoding Mechanisms: Mathematical Structure and Bias	5
2.3.1 Cryptographic Construction: The SHA-256 Architecture	5
2.3.2 Algebraic Construction: The Geometry of RS-ID	6
2.3.3 Structural Blindness and Randomization	7
2.4 Operational Prerequisite: Common Randomness Generation	7
2.5 Theoretical Limits and Algorithmic Trade-offs	8
2.5.1 The Algebraic Limit: Saturation and Field Size	8
2.5.2 The Cryptographic Perspective: Independence from Data Size	9
2.5.3 Engineering Trade-offs: Bound vs. Speed	9
2.6 Control-Theoretic Perspectives: Deadbands and Stability	9
2.6.1 K-ID as Deadband Control	9
2.6.2 Prevention of Zeno Behavior	10
2.6.3 Bounded-Input Bounded-Output (BIBO) Stability	10
2.7 Semantic Verification: Adapting to Control Flexibility	10
2.7.1 The Autonomous Driving Scenario	10
2.7.2 Human-Robot Collaboration and Jitter	11
2.7.3 Discussion: Trading Freshness for Value	11
3 Design Methodology	12
3.1 System Model and Data Flow	12
3.1.1 Baseline: Traditional Transmission Flow	12
3.1.2 Proposed: Identification (ID) Data Flow	13

3.2	Encoding Mechanisms and Reliability Analysis	13
3.2.1	Algebraic Implementation and Error Bound (RS-ID)	13
3.2.2	Cryptographic Construction and Error Analysis (SHA-256)	14
3.3	End-to-End Latency Model	14
3.4	Tolerance-Based Verification (K-ID)	15
3.5	Dynamic Evaluation Model: Random Walk with Drift	17
4	Implementation	18
4.1	Experimental Parameters	18
4.1.1	Evaluation Scenarios (Bandwidth Baselines)	18
4.1.2	LiDAR-Scale Scalability Parameters	19
4.2	Data Packet Structure Abstraction	19
4.3	Software Architecture	20
4.3.1	Modular Simulator Architecture	20
4.4	Latency Plot Generation	21
4.4.1	Logic	21
4.4.2	SHA-256	22
4.4.3	RS-ID	22
4.5	K-ID Parameter Sweep Setup	22
4.5.1	Model and Metrics	22
4.5.2	State Space and Quantization	22
4.5.3	Parameter Grid	23
4.5.4	Reproducibility	23
4.6	Dynamic Drift Experiment (Random Walk & Correction)	23
4.6.1	Parameterization	24
4.6.2	Reproducibility	24
5	Results and Evaluation	26
5.1	Empirical Parameter Verification	26
5.1.1	Encoding Latency (t_{ver})	26
5.1.2	Theoretical Reliability Limits	27
5.2	End-to-End Latency Evaluation	27
5.2.1	The Break-Even Analysis (Bandwidth Sensitivity)	28
5.2.2	Operational Bounds (Prediction Sensitivity)	28
5.2.3	LiDAR-Scale Payload Scalability	29
5.2.4	Overall Latency vs. Payload Size (Fixed n_{ver})	30
5.3	K-ID: Quantifying the Safety-Performance Trade-off	31
5.3.1	Mechanism Analysis	31
5.3.2	Pareto Optimization for Deployment	35
5.4	Dynamic Evaluation: Random Walk with Drift	38
5.4.1	Qualitative Behavior (Drift & Escape Time Series)	38
5.4.2	Quantitative Risk (Duration and Peak Drift)	39
6	Conclusion and Future Work	41
6.1	Conclusion	41
6.2	Future Work	42
6.2.1	Energy-Aware Identification Policies	42
6.2.2	Integration with 6G Physical Layer Security	42
6.2.3	Scalability to Multi-Agent Systems	42

6.2.4	Predictive Digital Twins and Adaptive K-ID	43
6.2.5	Decoder Compute, Scheduling, and Quantization Sensitivity	43
Bibliography		44
A Appendix		46

List of Figures

3.1	Data flow of Traditional Synchronization: The full state is transmitted regardless of the Digital Twin’s knowledge.	12
3.2	Data flow of ID Synchronization: High-bandwidth transmission is replaced by local encoding and conditional fallback.	13
3.3	K-ID synchronization flow. Compared to standard ID verification, the receiver accepts if the transmitted verifier is consistent with a tolerance set (rather than a single predicted state), and triggers fallback otherwise.	16
4.1	Abstract ID message format used for modeling. The normal path transmits only a verifier, while the fallback path transmits the full state payload. Fixed overhead fields are shown for completeness but are not explicitly modeled in the latency equations.	20
4.2	High-level simulator architecture used in the Python experiments. Encoder, Channel/Dynamics, and Decoder are decoupled so that coding schemes (SHA-256 vs. RS-ID), acceptance rules (ID vs. K-ID), and stochastic models (i.i.d. vs. time-correlated drift) can be varied independently.	21
5.1	End-to-end latency vs. channel bandwidth. The plots reveal the break-even point where the computational cost of Identification (ID) (t_{ver}) intersects with the transmission cost of the baseline. In the low-bandwidth regime ($B < 1$ Mbps), ID offers up to $10\times$ latency reduction.	28
5.2	Latency sensitivity to prediction error. ID relies on accurate predictions. For small payloads (a), the system must maintain $p_{desync} < 0.8$ to outperform the baseline. For large payloads (b), the transmission savings are so significant that ID remains superior even with poor predictions.	29
5.3	t_{ver} scalability vs. payload size at $n_{ver} = 16$ (log-log). Error bars show 95% CI half-width with relative CI controlled to $\leq 5\%$	30
5.4	Expected latency vs. payload size ($n_{ver} = 16$, $B = 5$ Mbps, $p_{desync} = 0.1$). The ID-based schemes incur a compute floor (t_{ver}) but can reduce communication cost to n_{ver}/B on non-fallback ticks.	31
5.5	Anatomy of the K-Identification (K-ID) trade-off (Gaussian error model). Parameters: $n_{data} = 4001$ bits, $B = 1$ Mbps, $t_{ver} = 2.13 \mu s$, 200,000 ticks per configuration, $K \in \{2, 5, 10, 20\}$, and $n_{ver} \in \{4, 8, 12, 16\}$. The results demonstrate the conflicting roles of tolerance (K) and verification stringency (n_{ver}). Increasing K improves performance (b, d) by ignoring minor errors, but raises collision risk (a). Increasing n_{ver} restores safety but adds transmission overhead.	32

5.6	Heatmap summary of the K-ID grid over (K, n_{ver}) . The left panel in each figure shows speedup; the right panel shows empirical overall risk $P(\text{unsafe} \cap \text{accept})$ (log-scaled).	32
5.7	Hidden DT-side cost of K-ID: acceptance-set size $ \mathcal{A}_K $ and distinct verifier count $ \mathcal{V}_K $ grow with tolerance K (here for $\Delta = 0.1$). This motivates treating K as a joint network-and-compute design parameter.	33
5.8	Quantization-resolution sensitivity (Δ). A finer Δ increases decoder cost $ \mathcal{A}_K $ (more hashes) and can shift the modeled overall-risk proxy. This highlights that Δ should be selected based on sensor precision and DT compute budget rather than treated as an arbitrary constant.	33
5.9	Anatomy of the K-ID trade-off (Uniform error model, variance-matched). Parameters: $n_{data} = 4001$ bits, $B = 1$ Mbps, $t_{ver} = 2.13 \mu s$, 200,000 ticks per configuration, $K \in \{2, 5, 10, 20\}$, and $n_{ver} \in \{4, 8, 12, 16\}$. The Uniform distribution is variance-matched to the Gaussian model.	34
5.10	Pareto frontier of K-ID design under two error models. The x-axis is the empirical <i>overall risk</i> $P(\text{unsafe} \cap \text{accept})$ per tick (i.e., miss events per tick), and the y-axis is speedup over traditional transmission. The solid line connects Pareto-nondominated configurations. Insets zoom into the low-risk region where points would otherwise overlap; the corresponding Pareto-optimal (K, n_{ver}) settings are listed in Table 5.4.	36
5.11	Drift-and-escape visualization for the dynamic random walk model ($K = 10$). The robot state S_t (blue) performs a random walk. The Digital Twin (DT) estimate $\hat{S}_t$ (orange, dashed) is held constant (Zero-Order Hold (ZOH) reference). The green band indicates the semantic safety set $\mathcal{A}_K$ and the red background the unsafe region. Green dots mark successful corrections after boundary breach; red crosses mark collision-induced misses where the boundary breach is not corrected, allowing the trajectory to continue diverging ("escape").	39
5.12	Dynamic risk vs. tolerance K (Gaussian drift). Increasing K reduces how often the boundary is checked (masking effect), but increases the miss proxy per verification event through the acceptance-set size. Short verifiers therefore yield longer boundary breaches and larger peak drift, even if the average per-tick miss probability remains small.	39
5.13	Empirical CDF of inter-transmission times (IFT) for the random-walk drift model. Larger K increases the silent intervals (masking), while smaller n_{ver} can shorten intervals by extending uncorrected excursions once a breach occurs. . .	40

List of Tables

5.1	Measured verification time t_{ver} (mean $\pm$ 95% CI half-width). Secure Hash Algorithm 256-bit (SHA-256) shows stable performance for fixed payloads, while Reed-Solomon Identification (RS-ID) is sensitive to the verifier length due to field arithmetic complexity.	27
5.2	Theoretical collision probability p_{miss} . RS-ID fails at short blocklengths ($n_{ver} = 4$) due to field saturation ($p_{miss} = 1.0$), whereas SHA-256 maintains robust collision resistance.	27
5.3	Measured verifier generation time t_{ver} for LiDAR-scale payload sizes ($n_{ver} = 16$). Values are mean $\pm$ 95% CI half-width; CI is controlled to $\leq 5\%$ relative error. .	30
5.4	Pareto-nondominated K-ID configurations under Gaussian and variance-matched Uniform models ($n_{data} = 4001$ bits, $B = 1$ Mbps). Overall risk is empirical $P(\text{unsafe} \cap \text{accept})$ per tick.	38

Acronyms

AoI	Age of Information	11
BIBO	Bounded-Input Bounded-Output	10
CPS	Cyber-Physical Systems	1
CR	Common Randomness	7
CSI	Channel State Information	42
DRBG	Deterministic Random Bit Generator	8
DT	Digital Twin	1–5, 7, 8, 10, 11, 15, 17, 23, 28, 38, 39, 41, 42, V
ETC	Event-Triggered Control	4, 5, 10
HRC	Human-Robot Collaboration	11
ID	Identification	1–3, 5, 7, 10, 11, 18, 19, 21, 26, 28–30, 41, 42, IV
IoT	Internet of Things	42
K-ID	K-Identification	1–3, 9–11, 15, 17–19, 26, 31, 32, 34–38, 41, 42, IV–VI
LiDAR	Light Detection and Ranging	29, 30, II
NCS	Networked Control System	9
PLKG	Physical Layer Key Generation	8, 42
PRNG	Pseudo-Random Number Generator	8, 42
PTP	Precision Time Protocol	8, 42
ROM	Random Oracle Model	14
RS-ID	Reed-Solomon Identification	2, 6–9, 19, 26, 27, 29, 30, 41, 42, VI
SHA-256	Secure Hash Algorithm 256-bit	2, 7, 9, 19, 26, 27, 29, 30, 41, 42, VI
TCP	Transmission Control Protocol	4
TDD	Time-Division Duplex	42
UDP	User Datagram Protocol	4
URLLC	Ultra-Reliable Low-Latency Communications	1, 3, 4, 41
VoI	Value of Information	11
ZOH	Zero-Order Hold	17, 23, 38, 39, 41, 43, V

1 Introduction

DTs have become a standard component in modern industrial automation for connecting physical assets with their virtual counterparts. By maintaining accurate virtual copies of systems like industrial robots, DTs enable real-time monitoring, predictive maintenance, and remote control. In time-critical Cyber-Physical Systems (CPS) applications, the utility of a DT is strictly defined by the synchronization accuracy between the physical robot and the digital model [21]. Consequently, ensuring state consistency typically demands Ultra-Reliable Low-Latency Communications (URLLC) capabilities to support high-frequency updates.

However, meeting these requirements via standard periodic transmission is resource-intensive. Since the DT often possesses a precise estimate of the robot’s state, continuously transmitting raw telemetry data creates significant redundancy. This inefficiency consumes scarce bandwidth without providing new information gain.

To mitigate this redundancy, recent research proposes *identification-based ID* synchronization [13]. Unlike classical Shannon-type communication, which aims to reconstruct the full message, ID schemes focus on a verification task: confirming whether the remote state matches a local hypothesis. Building on this concept, prior works [17] demonstrate that transmitting a compressed “verifier” (e.g., a short hash or randomized code) instead of the full state significantly reduces communication traffic. This approach effectively supports goal-oriented communication [6] in low-latency scenarios.

However, implementing ID synchronization in real-time control loops reveals two critical engineering trade-offs. First, regarding latency, the computational overhead of encoding on edge devices may exceed the time saved by reducing transmission traffic. This shift from a communication-bound to a computation-bound bottleneck necessitates a rigorous end-to-end feasibility analysis. Second, reliability becomes a primary concern when moving from exact to semantic verification. In practical control, enforcing bitwise synchronization (standard ID) is often overly conservative, triggering unnecessary updates for benign deviations such as sensor noise. To address this, we extend identification to semantic “safe ranges”, where the system verifies set membership rather than a unique value. This relaxation, however, introduces collision risks, specifically Type II (miss) errors, where dangerous desynchronizations are incorrectly classified as safe. Therefore, quantifying the relationship between the semantic acceptance range and safety risks is essential.

To address these limitations, this thesis presents a complementary study to existing ID frameworks, extending the analysis from pure traffic optimization to an integrated view of latency and reliability in K-ID systems. The main contributions are:

First, we develop an end-to-end latency model combining computational and transmission delays. We compare cryptographic (SHA-256) and algebraic (RS-ID) encoding schemes to determine the quantitative thresholds (break-even points) where ID yields a net latency benefit as a function of bandwidth and verifier length.

Second, we formalize the K-ID problem for robotic monitoring, distinguishing *safe* from *unsafe* desynchronization via semantic safety sets. We derive quantitative bounds on miss error probabilities induced by K-ID mechanisms.

Third, we conduct a systematic evaluation using a reproducible software stack. Through Monte Carlo experiments on Gaussian robot trajectories, we expose a non-monotonic dependence between the semantic acceptance range (Set B) and error rates. These results provide design guidelines for balancing semantic tolerance with safety detection in edge-based DTs networks.

Relation to prior work. While ID rests on a solid information-theoretic foundation, engineering studies have primarily focused on traffic suppression. We extend this line of work by treating latency as a primary metric alongside traffic and by generalizing from single identification to K-ID, which aligns better with real-world DT workflows. Our results aim to inform design choices regarding *when to use ID/K-ID and with which verifier lengths* under realistic bandwidth conditions.

Organization. Chapter 2 introduces the background and related work; Chapter 3 details the system model and measurement protocol; Chapter 4 describes the software stack and reproducibility; Chapter 5 presents the experimental results, including latency–traffic curves parameterized by bandwidth and load; Chapter 6 concludes and outlines future engineering directions for K-ID.

2 Background and Related Work

This chapter reviews the transition from classical data transmission to identification-based synchronization, positioning it within the broader context of semantic communication [22] for 6G networks. We analyze alternative synchronization strategies—such as differential encoding and event-triggered control—to motivate the engineering need for the identification (ID) paradigm. We then compare practical encoding mechanisms by contrasting the avalanche dynamics of cryptographic hashing with the geometric properties of algebraic coding. Finally, we discuss how this paradigm generalizes to K-ID, where strict equality constraints are relaxed to semantic safety sets to accommodate natural variability in physical control loops.

2.1 From Data Reconstruction to Semantic Identification

In time-critical control loops, the utility of a DT depends on its synchronization with the physical asset. Conventional protocols treat this as a data reconstruction problem, where the robot periodically transmits its full state vector S_t (e.g., joint angles and velocities) to the DT. While robust, this approach overlooks a key redundancy: the DT, using its internal kinematic model, already maintains a prediction $\hat{S}_t$ of the robot’s state. The communication task is therefore not to inform the DT “where the robot is,” but to verify “whether the DT’s prediction is correct.”

This shift aligns with the paradigm of *semantic communication* [19] in 6G networks. Unlike classical Shannon-type communication, which emphasizes accurate reconstruction, semantic communication emphasizes information that matters for the receiver’s goal. In the context of URLLC, this suggests prioritizing “actionable” information—such as safety violations—over raw telemetry.

Accordingly, the coding objective shifts from Shannon’s transmission capacity to the identification capacity defined by Ahlswede and Dueck [2]. In transmission, each possible state must map to a disjoint decoding set, limiting the number of distinguishable states to 2^{nR} for blocklength n . In contrast, identification permits overlaps between decoding sets, as long as pairwise intersections are statistically negligible. This relaxation enables a doubly exponential growth in identifiable states ($2^{2^{nR}}$), meaning that short verifiers can discriminate among an extremely large state space while allowing a vanishingly small error probability.

2.2 Alternative Synchronization Strategies

Before detailing the implementation of identification, it is necessary to rigorously evaluate why established data reduction strategies—specifically Delta Encoding and Event-Triggered Control—are often insufficient for unreliable edge networks.

2.2.1 Full State Transmission (The Baseline)

The standard baseline in industrial automation is the periodic transmission of the full state vector S_t . This approach is favored for its statelessness; each packet is self-contained. If a packet at time t is lost due to channel interference, the packet at $t + 1$ allows the DT to immediately recover the correct state. However, this robustness comes at the cost of extreme bandwidth inefficiency. In steady-state operations, where $S_t \approx S_{t-1}$, the information entropy of the transmitted packet is near zero, yet it consumes the full bandwidth allocated to the payload n_{data} .

2.2.2 Delta Encoding and the Drift Problem

To address this redundancy, Delta Encoding [15] (or differential transmission) is frequently employed. This method transmits only the difference vector $\Delta_t = S_t - S_{t-1}$. Since Δ_t typically has a smaller dynamic range than the absolute state S_t , it can be quantized with fewer bits, offering significant compression ratios.

However, Delta Encoding introduces a key vulnerability: *statefulness*. The reconstruction of the current state at the receiver, denoted as S'_t , depends recursively on the history of received updates: $S'_t = S'_{t-1} + \Delta_t$. This recursive dependency creates a clear failure mode in unreliable transport protocols like User Datagram Protocol (UDP). A single lost packet Δ_k causes a persistent offset between the physical robot and the DT, a phenomenon known as “state drift.” [11] While reliable protocols like Transmission Control Protocol (TCP) can prevent packet loss, retransmissions introduce non-deterministic latency jitter, which can violate the timing deadlines of real-time control loops. Therefore, Delta Encoding poses a high risk for URLLC applications.

2.2.3 Event-Triggered Control (ETC)

Another prominent alternative is Event-Triggered Control (Event-Triggered Control (ETC)). Unlike Time-Triggered (periodic) systems, ETC only transmits data when the deviation between the current state and the last transmitted state exceeds a threshold δ : $\|S_t - S_{last_sent}\| > \delta$ [20]. This approach effectively silences the network during periods of inactivity.

Nevertheless, ETC relies fundamentally on a “push” communication model. A silent channel is ambiguous to the receiver: it could indicate that the system is operating safely within bounds, or it could indicate that the sensor has failed or the link has been severed. In safety-critical robotics, such ambiguity is often unacceptable. Furthermore, ETC requires the sensing node to perform continuous monitoring and threshold checking, pushing computational complexity to the energy-constrained edge [7].

2.2.4 The Identification Advantage

In contrast to these alternatives, the **ID** scheme proposed in this thesis combines statelessness with strong compression. Unlike Delta Encoding, **ID** verification checks the absolute state S_t directly and is therefore immune to drift; if a packet is lost, the next packet again provides an independent check. Unlike **ETC**, **ID** supports a “verify” model, allowing the **DT** to actively confirm synchronization with minimal overhead. This combination is well-suited for maintaining high-fidelity **DTs** over unreliable networks.

2.3 Encoding Mechanisms: Mathematical Structure and Bias

To realize the theoretical capacity of identification in edge computing, the encoding process must map a high-dimensional state space $\mathcal{S}$ into a low-dimensional verifier space $\mathcal{V}$ while minimizing collision probabilities. This section dissects the internal mechanisms of the two candidate algorithms—cryptographic hashing (SHA-256) and algebraic coding (RS-ID)—to reveal the mathematical origins of their reliability.

2.3.1 Cryptographic Construction: The SHA-256 Architecture

To understand the source of the high entropy in the generated verifiers, it is necessary to examine the internal structure of the SHA-256 algorithm [16]. SHA-256 belongs to the SHA-2 family and is built upon the Merkle-Damgård construction, which iteratively processes the input state S using a dedicated compression function. The process consists of three mathematically distinct stages: preprocessing, message expansion, and the state update loop.

Preprocessing and Padding

The input message M (comprising the state S concatenated with the seed r) is first padded to ensure its length is a multiple of 512 bits. The padding scheme appends a single ‘1’ bit, followed by k zero bits, and finally a 64-bit integer representing the original message length. This padded message is parsed into N blocks of 512 bits, denoted as $M^{(1)}, M^{(2)}, \dots, M^{(N)}$.

Message Schedule (Expansion)

The core of the diffusion process begins with the Message Schedule. Each 512-bit block $M^{(i)}$ is expanded from 16 words (32-bit each) into 64 words, denoted as $W_0, \dots, W_{63}$. The first 16 words are copied directly from the message block. The remaining 48 words are generated recursively, creating a complex dependency chain where every future bit depends on multiple past bits:

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16} \pmod{2^{32}} \quad (2.1)$$

where the small sigma functions σ_0 and σ_1 introduce the initial bitwise diffusion via rotation (ROTR) and shifting (SHR):

$$\sigma_0(x) = \text{ROTR}^7(x) \oplus \text{ROTR}^{18}(x) \oplus \text{SHR}^3(x) \quad (2.2)$$

$$\sigma_1(x) = \text{ROTR}^{17}(x) \oplus \text{ROTR}^{19}(x) \oplus \text{SHR}^{10}(x) \quad (2.3)$$

This expansion ensures that a single bit flip in the input $M^{(i)}$ propagates to affect a vast majority of the 64 words in the schedule, setting the stage for the avalanche effect.

The Compression Loop

The 64 expanded words W_t are then fed into the compression function. The working state consists of eight 32-bit variables, labeled a through h , initialized to constant prime-root values. For each step t from 0 to 63, the state is updated using two temporary variables T_1 and T_2 :

$$T_1 = h + \Sigma_1(e) + Ch(e, f, g) + K_t + W_t \quad (2.4)$$

$$T_2 = \Sigma_0(a) + Maj(a, b, c) \quad (2.5)$$

Here, K_t represents 64 constant 32-bit words. The non-linear logical functions Ch (Choose) and Maj (Majority), along with the big sigma functions Σ_0 and Σ_1 , serve as the mixing engine:

$$Ch(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z) \quad (2.6)$$

$$Maj(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z) \quad (2.7)$$

$$\Sigma_0(x) = ROTR^2(x) \oplus ROTR^{13}(x) \oplus ROTR^{22}(x) \quad (2.8)$$

$$\Sigma_1(x) = ROTR^6(x) \oplus ROTR^{11}(x) \oplus ROTR^{25}(x) \quad (2.9)$$

After calculating T_1 and T_2 , the registers are shifted ($h \leftarrow g, g \leftarrow f, \dots$), and updated as:

$$e \leftarrow d + T_1, \quad a \leftarrow T_1 + T_2 \quad (2.10)$$

This rigorous mixing guarantees that after 64 rounds, every bit of the final hash output is a complex, non-linear function of every bit of the input block. Finally, the output verifier v is obtained by truncating this digest to the target length n_{ver} , retaining the high-entropy properties generated by this procedure.

2.3.2 Algebraic Construction: The Geometry of RS-ID

In contrast to the bitwise diffusion of SHA-256, [RS-ID](#) relies on the geometric properties of polynomials over finite fields[18].

Data as a Polynomial Curve

Let the state vector S be decomposed into k segments $\{s_0, s_1, \dots, s_{k-1}\}$, where each segment is an element of the Galois Field $\mathbb{F}_q$. We interpret these segments not as raw data, but as coefficients of a unique polynomial curve $P_S(x)$ of degree $k - 1$:

$$P_S(x) = s_0 + s_1x + s_2x^2 + \dots + s_{k-1}x^{k-1} = \sum_{i=0}^{k-1} s_i x^i \quad (2.11)$$

The generation of a verifier v using a random seed r is geometrically equivalent to *sampling this curve* at the coordinate $x = r$:

$$v = P_S(r) \pmod{q} \quad (2.12)$$

Collision as Intersection

A collision (Type II error) occurs when the true state S and the predicted state $\hat{S}$ are distinct ($S \neq \hat{S}$), yet their verifiers match ($v = \hat{v}$). In our geometric model, this implies:

$$P_S(r) = P_{\hat{S}}(r) \implies P_S(r) - P_{\hat{S}}(r) = 0 \quad (2.13)$$

Let $\Delta(x) = P_S(x) - P_{\hat{S}}(x)$ be the difference polynomial. Since $S \neq \hat{S}$, $\Delta(x)$ is a non-zero polynomial with degree at most $k - 1$. By the *Fundamental Theorem of Algebra*, a non-zero polynomial of degree d can have at most d roots.

This geometric constraint provides the rigorous upper bound for the collision probability. The number of "bad seeds" r that cause a collision is limited to the number of intersection points between the two curves $P_S(x)$ and $P_{\hat{S}}(x)$, which is at most $k - 1$. Thus, for a field of size q , the probability is:

$$P_{\text{miss}} = \frac{\text{Number of Roots}}{q} \leq \frac{k - 1}{q} \quad (2.14)$$

This derivation visualizes the identification process as "checking for intersection." Since two distinct curves cannot intersect everywhere, checking a random point r effectively distinguishes them with high probability [18].

2.3.3 Structural Blindness and Randomization

A critical limitation of deterministic compression is "structural blindness." Consider a simplified case where the encoder uses a modulo operation. If the error $e = S - \hat{S}$ happens to be a multiple of the modulus, the generated verifiers will be identical ($v = \hat{v}$) regardless of the magnitude of the error. Even if the robot's state S changes continuously, this error pattern might persist, leading to a persistent Type II (miss) error.

To eliminate structural blindness, identification theory mandates the use of *common randomness*—a shared seed r that changes with each transmission. In [RS-ID](#), changing r corresponds to sampling the polynomial at a different x -coordinate. In [SHA-256](#), appending a time-varying salt ($S||r$) changes the avalanche dynamics. By varying r , the mapping changes at every time step; although a collision may occur at a specific seed r_1 , the probability that the same error causes a collision at a new seed r_2 is independent. This converts potential "permanent failures" into "transient probabilistic events."

2.4 Operational Prerequisite: Common Randomness Generation

As established in Section 2.3, the theoretical validity of identification codes hinges on the availability of [Common Randomness \(CR\)](#)—a shared source of entropy between the encoder (robot) and the decoder ([DT](#)). Without a dynamic shared seed r , the system reduces to a deterministic mapping and becomes susceptible to structural blindness, where persistent error patterns can remain undetectable [13]. Therefore, the practical deployment of [ID](#) requires a robust mechanism to generate or synchronize r without negating the bandwidth savings.

The most straightforward approach to establish this shared entropy is to use synchronized **Pseudo-Random Number Generators (PRNGs)**, formally standardized as **Deterministic Random Bit Generators (DRBGs)** [4]. By initializing identical generators with a pre-shared secret key and tying them to a synchronized time index—typically maintained via the **Precision Time Protocol (PTP)** [10]—both the robot and the **DT** can locally derive an identical sequence of seeds r_t for each time slot. While this method has zero communication overhead during steady-state operation, it introduces a dependency on clock synchronization. Any drift in the robot’s clock relative to the **DT** results in a seed mismatch ($r_{robot} \neq r_{DT}$), causing verifier checks to fail continuously. This loss of synchronization can manifest as a sustained burst of false positives, forcing the system into a fallback mode until resynchronization.

Alternative approaches, such as **Physical Layer Key Generation (PLKG)**, attempt to extract randomness from channel reciprocity in wireless networks [14]. While **PLKG** aligns with physical-layer security paradigms by exploiting channel noise entropy [1], it often requires additional reconciliation overhead to correct quantization mismatches. Consequently, to isolate the performance characteristics of the encoding mechanisms, this thesis assumes an idealized PRNG-based synchronization model for the experimental evaluation.

2.5 Theoretical Limits and Algorithmic Trade-offs

While the introduction of randomization allows for drastic compression, the output length cannot be reduced arbitrarily. The choice between algebraic (RS-ID) and cryptographic (SHA-256) implementations is governed by a fundamental tension between theoretical error bounds and computational efficiency.

2.5.1 The Algebraic Limit: Saturation and Field Size

In the polynomial model of **RS-ID**, the verifier length L is constrained by the algebraic properties of the state representation. Since a state vector of length k is interpreted as a polynomial of degree $k - 1$, two distinct state polynomials can intersect at most at $k - 1$ points. To ensure reliable identification, the size of the evaluation field $Q = 2^L$ must be significantly larger than the number of potential intersection points. If $Q < k$, the “collision points” can saturate the field, making it mathematically impossible to find a seed r that distinguishes the two states via a single sample.

Therefore, a necessary condition for the reliability of algebraic identification is that the field size must grow with the data size:

$$2^L > k \quad \Rightarrow \quad L > \log_2 k \quad (2.15)$$

This inequality represents the physical manifestation of the double exponential limit established by Ahlswede and Dueck [2]. It dictates that for **RS-ID**, the verifier length L must scale logarithmically with the data size k . As long as this condition is met, **RS-ID** provides a rigorous upper bound λ_2 on the collision probability, ensuring that no pair of states exceeds a worst-case error threshold regardless of the input distribution [13].

2.5.2 The Cryptographic Perspective: Independence from Data Size

In contrast to the algebraic constraints, cryptographic methods like [SHA-256](#) operate under the Random Oracle Model. The collision probability of a truncated hash is approximately 2^{-L} , which is invariant to the input data size k . Unlike [RS-ID](#), [SHA-256](#) does not suffer from “saturation” when the input data grows. A 16-bit hash verifier has the same uniform collision probability whether verifying a 100-byte state or a 1-megabyte state. This makes cryptographic approaches robust for large-scale data vectors where satisfying $2^L > k$ might require an impractically large field for [RS-ID](#).

2.5.3 Engineering Trade-offs: Bound vs. Speed

The selection of the encoding mechanism involves a critical engineering trade-off between provable safety and computational latency. Although [RS-ID](#) offers unconditional security guarantees via the λ_2 bound, its computational cost is significant. Empirical studies demonstrate that [RS-ID](#) encoding involves complex finite field arithmetic, resulting in computation times that are orders of magnitude higher than hardware-accelerated hash functions [13].

For real-time control loops operating at short block lengths, the theoretical advantages of [RS-ID](#) often do not translate into measurable practical gains, as the hash function’s collision rate is sufficiently low for uniform source distributions. However, for mission-critical safety systems where worst-case adversarial distributions must be mitigated, the mathematical guarantee of [RS-ID](#) remains indispensable. Consequently, this thesis primarily adopts the cryptographic approach for its latency benefits in edge computing, while acknowledging [RS-ID](#) as the theoretical benchmark for safety-critical bounds.

2.6 Control-Theoretic Perspectives: Deadbands and Stability

Beyond communication efficiency, the deployment of [K-ID](#) changes the control loop structure. From a control-theoretic perspective, [K-ID](#) yields a [Networked Control System \(NCS\)](#) operating under a semantic sampling policy. This section analyzes the implications for stability and execution semantics, and maps [K-ID](#) parameters to established control concepts.

2.6.1 K-ID as Deadband Control

The tolerance mechanism in [K-ID](#), defined by the acceptance set $\mathcal{B}$, is mathematically equivalent to *Deadband Control* (or Dead-Zone Control) in non-linear systems theory. In classical mechanics, deadbands are often viewed as parasitic nonlinearities (e.g., gear backlash) that degrade performance. However, in resource-constrained [NCS](#), artificially introduced deadbands are a recognized strategy for reducing actuator wear and communication load [3].

By enforcing a “silence” policy when the synchronization error $e_t = S_t - \hat{S}_t$ satisfies $|e_t| \leq K$, [K-ID](#) implements *Lebesgue Sampling* (sampling based on value domains), as opposed to the traditional Riemann Sampling (sampling based on time domains). This semantic filtering prevents the control loop from reacting to high-frequency, low-amplitude noise (chattering), thereby reducing mechanical stress on physical actuators while preserving bandwidth for significant trajectory corrections.

2.6.2 Prevention of Zeno Behavior

A pervasive challenge in standard Event-Triggered Control (ETC) is the risk of *Zeno behavior*—a phenomenon where the inter-event time converges to zero, causing the system to trigger an infinite number of updates in a finite time interval [7]. This can occur when the state trajectory “slides” along the triggering threshold, potentially leading to computational livelock or network storms.

K-ID inherently circumvents this instability through its hybrid architecture. Although the *decision* to update is event-driven (based on the semantic check $v \in \mathcal{V}_K$), the *verification opportunity* occurs at discrete, time-triggered intervals (ticks). This imposes a strict lower bound on the inter-transmission time, denoted as $T_{min} \geq T_{tick}$. By structural design, K-ID guarantees *Zeno-freeness*, ensuring that the network load remains deterministically bounded even in worst-case divergence scenarios.

2.6.3 Bounded-Input Bounded-Output (BIBO) Stability

The introduction of the semantic tolerance K raises the question of closed-loop stability. Under K-ID, the DT (controller) operates on an estimated state $\hat{S}_t$ that deviates from the true plant state S_t by an error term δ_t . Assuming the absence of collision-induced miss errors (which are probabilistically negligible as shown in Section 2.3), the synchronization error is strictly bounded:

$$\|\delta_t\| = \|S_t - \hat{S}_t\| \leq K \quad (2.16)$$

From a stability analysis perspective, this bounded error acts as a bounded disturbance input to the controller. According to the principle of Bounded-Input Bounded-Output (BIBO) stability [9], as long as the underlying control law can reject disturbances of magnitude K , the closed-loop system remains stable. This implies that the design of K is not only a traffic parameter but also a stability constraint: K must remain within the controller’s stability margin (or region of attraction).

2.7 Semantic Verification: Adapting to Control Flexibility

While standard identification provides a rigorous method for exact state verification, its binary nature—accepting only a single, precise value—often contradicts the operational flexibility required in modern robotic control. In many practical scenarios, the DT does not need to enforce a strict equality constraint ($x = \hat{x}$) but rather needs to guarantee that the robot operates within a permissible semantic range. This nuance necessitates the adoption of K-ID, which generalizes the verification target from a unique point to a semantic safety set.

2.7.1 The Autonomous Driving Scenario

Consider a practical example of an autonomous robot (or vehicle) navigating a public roadway. The control policy might specify a safe velocity corridor, such as “maintain speed between 40 km/h and 60 km/h.” If the robot moves at 45 km/h while the DT predicted 50 km/h, a standard ID scheme would flag this as a collision (mismatch) because the hashed values of “45”

and “50” are distinct. This triggers an unnecessary data transmission to correct a behavior that is semantically valid.

K-ID resolves this inefficiency by defining the acceptance region $\mathcal{B}$ as the interval $[40, 60]$. As long as the robot’s state falls within this set, the **K-ID** verifier confirms validity without consuming bandwidth for minor deviations.

2.7.2 Human-Robot Collaboration and Jitter

This logic extends beyond simple velocity constraints to more complex spatial and interactive scenarios. For instance, in **Human-Robot Collaboration (HRC)**, the exact coordinates of a robotic arm are less critical than ensuring the end-effector remains outside a defined “human safety bubble” [21]. Similarly, for high-precision assembly tasks involving compliant mechanisms, the robot may exhibit minor high-frequency vibrations (jitter) that are mechanically acceptable. Standard **ID** would treat these micro-vibrations as errors, whereas **K-ID** can be tuned to ignore high-frequency noise while still detecting significant trajectory deviations.

By shifting the verification paradigm from data exactness to semantic compliance, **K-ID** aligns the synchronization protocol with the actual control objectives. It allows the system to filter out benign variability—whether it be speed fluctuations, sensor noise, or compliant jitter—and focus limited communication resources on detecting genuine safety violations. This thesis specifically investigates the implementation of such semantic verification using randomized encoding on edge constraints.

2.7.3 Discussion: Trading Freshness for Value

The scenarios described above illustrate a deviation from traditional network optimization. Conventional real-time protocols often aim to minimize the **Age of Information (AoI)** [12], keeping the **DT**’s estimate as fresh as possible regardless of content. Under an **AoI**-minimization strategy, the robot would transmit its velocity continuously even when cruising steadily within the safe corridor, purely to prevent the timestamp from “aging.”

K-ID challenges this paradigm by introducing a semantic filter that prioritizes the **Value of Information (VoI)** over simple freshness. As demonstrated in the autonomous driving case, a “stale” position estimate (high **AoI**) that remains within the safety tolerance $\mathcal{B}$ retains its utility for the control loop. By allowing **AoI** to increase during semantically safe intervals, **K-ID** trades unnecessary freshness for bandwidth efficiency. This is consistent with goal-oriented communication [19], where the objective shifts from minimizing average latency to minimizing the latency of *meaningful* (unsafe) updates.

3 Design Methodology

This chapter formulates the system model for identification-based synchronization and derives the theoretical bounds for its reliability and latency. We contrast the data flow of classical transmission with the proposed identification scheme, explicitly breaking down the operational stages from state generation to synchronization. We then quantify the collision probabilities and end-to-end latency based on these distinct workflows.

3.1 System Model and Data Flow

We consider a networked control system where a Robot (Sender) synchronizes its state S_t with a Digital Twin (Receiver) maintaining a prediction $\hat{S}_t$. The fundamental difference between the two approaches lies in their data processing pipelines.

3.1.1 Baseline: Traditional Transmission Flow

The standard synchronization protocol follows a linear, “always-on” approach. As illustrated in Figure 3.1, the robot first samples its sensors to generate the full state message S_t (size n_{data}) and transmits the entire payload over the channel. The Digital Twin receives the message, overwrites its local prediction $\hat{S}_t$ with the received S_t , and confirms the update.

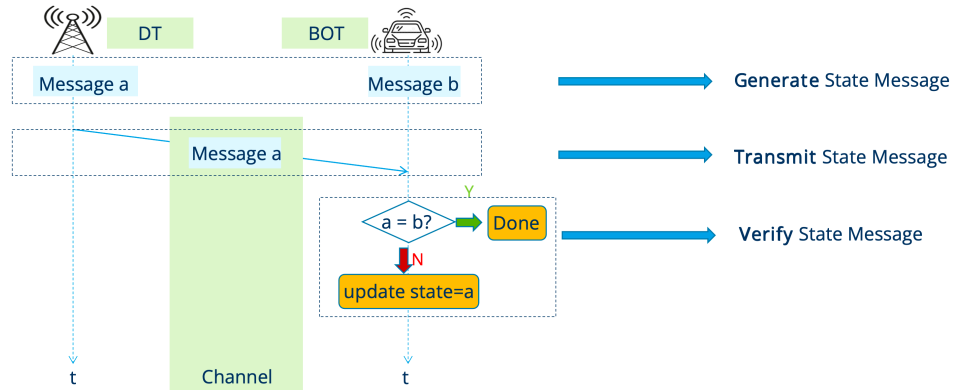


Figure 3.1: Data flow of Traditional Synchronization: The full state is transmitted regardless of the Digital Twin’s knowledge.

This process is robust but inefficient, since the transmission cost is paid unconditionally. Under this baseline, the transmission latency is deterministic and scales directly with the state size, $T_{trans} = n_{data}/B$.

3.1.2 Proposed: Identification (ID) Data Flow

The identification-based protocol reduces communication load by verifying consistency before sending the full state. As shown in Figure 3.2, both the Robot and the DT first generate their respective state views, S_t and $\hat{S}_t$. The Robot computes a compact verifier $v = E(S_t, r)$ using a shared seed r , while the DT computes the corresponding reference $\hat{v} = E(\hat{S}_t, r)$ locally. Only the short verifier v (size $n_{ver} \ll n_{data}$) is transmitted. The DT then compares v and $\hat{v}$: if they match, the protocol terminates; if they differ, the DT triggers fallback and requests the full payload S_t to resynchronize.

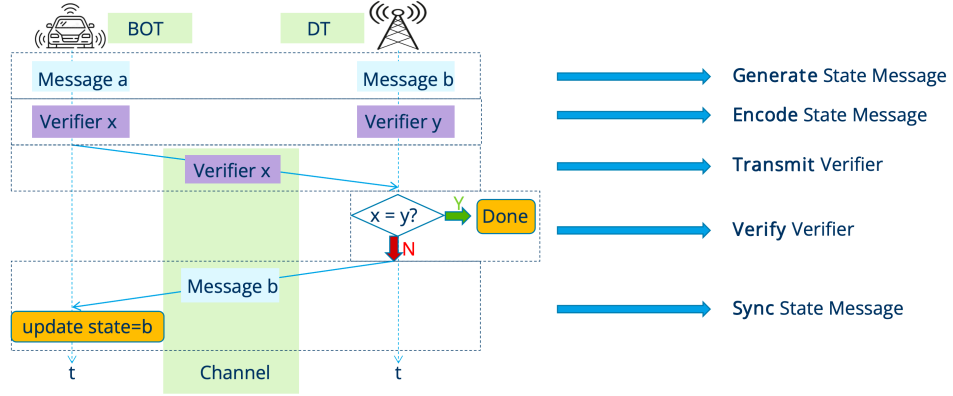


Figure 3.2: Data flow of ID Synchronization: High-bandwidth transmission is replaced by local encoding and conditional fallback.

3.2 Encoding Mechanisms and Reliability Analysis

The safety of the ID flow depends on the reliability of the *Verify* stage. A critical failure occurs if the states differ ($S_t \neq \hat{S}_t$) but their verifiers collide ($v = \hat{v}$), defined as a Type II Error (p_{miss}). We investigate two distinct encoding mechanisms, detailing their construction and theoretical error bounds.

3.2.1 Algebraic Implementation and Error Bound (RS-ID)

To integrate the RS-ID scheme into our synchronization protocol, we map the high-dimensional state vector S of length n_{data} to a polynomial over the finite field $\mathbb{F}_q$, where $q = 2^{n_{ver}}$.

Following the construction detailed in Section 2.3.2, the bitstream S is sliced into k coefficients. For the specific bandwidth constraints of our system model, the number of coefficients is determined by:

$$k = \left\lceil \frac{n_{data}}{n_{ver}} \right\rceil \quad (3.1)$$

The system generates the verifier v by evaluating the polynomial $P_S(x) = \sum_{i=0}^{k-1} s_i x^i$ at the shared seed r .

For the experimental evaluation and latency modeling, we apply the theoretical upper bound derived in Chapter 2 (Schwartz-Zippel Lemma). The probability of a Type II error (collision) is strictly modeled as:

$$P_{miss}^{RS} \leq \frac{k}{q} = \frac{\lceil n_{data}/n_{ver} \rceil}{2^{n_{ver}}} \quad (3.2)$$

This equation serves as the reliability constraint for the design: to ensure robust identification ($P_{miss} \ll 1$), the verifier length n_{ver} is selected such that the field size exponentially exceeds the data complexity k .

3.2.2 Cryptographic Construction and Error Analysis (SHA-256)

Unlike the algebraic structure of RS-ID, the security of hash-based identification relies on computational hardness assumptions. We verify the state using a truncated cryptographic hash function (SHA-256):

$$v = \text{Truncate}(\text{SHA-256}(S||r), n_{ver}) \quad (3.3)$$

where $||$ denotes concatenation with the random seed r .

To analyze the collision probability theoretically, we adopt the *Random Oracle Model (ROM)* paradigm introduced by Bellare and Rogaway [5]. In this model, the hash function $H(\cdot)$ is idealized as a mathematical oracle that maps every unique input query to a response selected *uniformly and independently* from the output domain $\{0, 1\}^{n_{ver}}$.

Under the ROM, for any robot state S , the hash output $H(S||r)$ is a random variable following a uniform distribution over the verifier space of size $N = 2^{n_{ver}}$. Given a pre-calculated verifier $\hat{v}$ from the Digital Twin, the probability of a collision (Type II error) is equivalent to the probability that a uniformly selected random value hits a specific target in the sample space.

$$P_{miss}^{SHA} = \Pr[H(S||r) = \hat{v}] = \frac{1}{|\{0, 1\}^{n_{ver}}|} = \frac{1}{2^{n_{ver}}} \quad (3.4)$$

This derivation relies on the property that in a uniform distribution, the probability of any single outcome is the reciprocal of the sample space size.

Comparing Equation (3.2) with Equation (3.4) highlights a key structural difference. In RS-ID, the error probability scales linearly with the data size ($k \approx n_{data}/n_{ver}$), reflecting the geometric fact that higher-degree polynomials admit more roots. Under the Random Oracle assumption, the SHA-256 collision probability is invariant to n_{data} . As a result, the cryptographic approach remains well-behaved for large state vectors (e.g., LiDAR point clouds), but it replaces the unconditional Schwartz-Zippel guarantee with a computational idealization.

3.3 End-to-End Latency Model

Based on the data flows defined in Section 3.1, we mathematically formulate the expected latency.

Since the traditional flow always executes the full transmission, its latency is deterministic:

$$T_{trad} = \frac{n_{data}}{B} \quad (3.5)$$

where B is the channel bandwidth.

In contrast, the ID flow is probabilistic. The total time includes the mandatory encoding and verifier transmission, plus the conditional penalty for fallback. Matching the stages in Figure 3.2:

$$T_{ID} = \underbrace{t_{ver}}_{\text{Encode}} + \underbrace{\frac{n_{ver}}{B}}_{\text{Transmit Verifier}} + \underbrace{p_{desync}(1 - p_{miss})\frac{n_{data}}{B}}_{\text{Sync (Fallback Penalty)}} \quad (3.6)$$

Here, t_{ver} denotes the computation time of the Encode stage, p_{desync} is the probability that the DT prediction fails ($S_t \neq \hat{S}_t$), and the fallback penalty $(1 - p_{miss})\frac{n_{data}}{B}$ is incurred only when a desynchronization exists and is successfully detected by the verifier.

This model highlights the fundamental trade-off: ID reduces the transmission term from $\frac{n_{data}}{B}$ to $\frac{n_{ver}}{B}$ but adds a computational cost t_{ver} . Therefore, ID is advantageous only when the bandwidth B is limited (making transmission expensive) or when the prediction accuracy is high (keeping p_{desync} small).

3.4 Tolerance-Based Verification (K-ID)

The ID data flow described in Section 3.1 assumes a strict equality check between a single predicted state $\hat{S}_t$ and the true state S_t . In practice, however, many cyber-physical systems can tolerate bounded deviations without requiring a full resynchronization. We therefore extend the verification stage to a *tolerance-based* rule, referred to as **K-ID**.

Figure 3.3 illustrates the resulting protocol. The overall stages remain identical to the ID synchronization flow (state generation, encoding, and transmission of a short verifier, followed by a local verification step and conditional fallback). The only difference is the decision criterion at the receiver: instead of checking equality against a single predicted verifier $\hat{v}$, the **DT** accepts if the received verifier matches *any* verifier in a precomputed set corresponding to a tolerance neighborhood around $\hat{S}_t$.

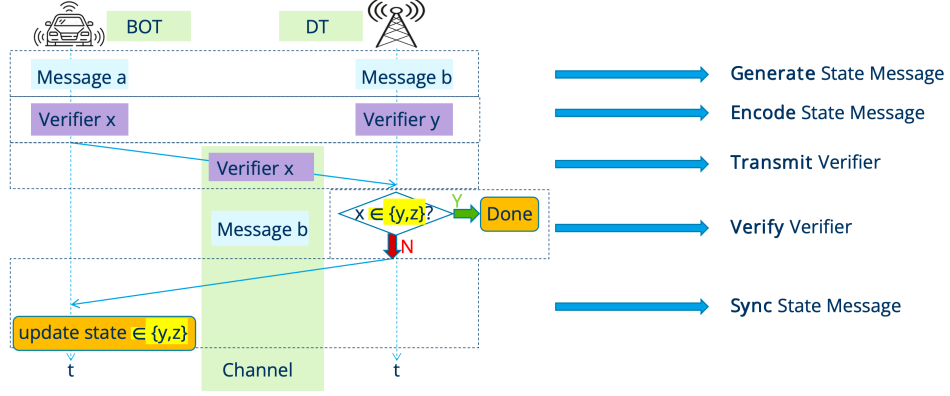


Figure 3.3: K-ID synchronization flow. Compared to standard ID verification, the receiver accepts if the transmitted verifier is consistent with a tolerance set (rather than a single predicted state), and triggers fallback otherwise.

Given the DT prediction $\hat{S}_t$ and a tolerance radius K , the DT defines an acceptable set

$$\mathcal{A}_K(\hat{S}_t) = \{s \mid \|s - \hat{S}_t\| \leq K\}. \quad (3.7)$$

In the experiments in this thesis, we use a scalar state for clarity (so $\|\cdot\|$ is absolute value), but the K-ID logic generalizes to vector-valued states with an appropriate norm.

The Robot computes the verifier using the same truncated SHA-256 construction as in Equation (3.3), with the current state S_t substituted for S . The DT pre-computes the set of valid verifiers for the acceptable states,

$$\mathcal{V}_K = \{\text{Truncate}(\text{SHA256}(s\|r), n_{\text{ver}}) \mid s \in \mathcal{A}_K(\hat{S}_t)\}, \quad (3.8)$$

and *accepts* if $v \in \mathcal{V}_K$. Otherwise, it triggers fallback and requests the full payload.

Define an *unsafe* event as the true state lying outside the tolerance region, i.e., $S_t \notin \mathcal{A}_K(\hat{S}_t)$. A miss occurs when the DT accepts despite being unsafe:

$$p_{\text{miss},K} = \Pr[v \in \mathcal{V}_K \mid S_t \notin \mathcal{A}_K(\hat{S}_t)]. \quad (3.9)$$

Under the Random Oracle assumption used in Section 3.2.2, the verifier v for an unsafe state is uniform over $\{0, 1\}^{n_{\text{ver}}}$. Hence, the conditional miss probability is bounded by the fraction of verifier space covered by acceptable hashes:

$$p_{\text{miss},K}^{\text{SHA}} \approx \frac{|\mathcal{V}_K|}{2^{n_{\text{ver}}}} \leq \frac{|\mathcal{A}_K(\hat{S}_t)|}{2^{n_{\text{ver}}}}. \quad (3.10)$$

This highlights a key trade-off: increasing K enlarges the acceptable set (reducing fallback frequency), but also increases the miss probability conditional on being unsafe.

To connect the tolerance rule to both safety and latency, let $p_{\text{unsafe}}(K) = \Pr[S_t \notin \mathcal{A}_K(\hat{S}_t)]$. If the prediction error $e = S_t - \hat{S}_t$ is modeled as $\mathcal{N}(0, \sigma^2)$, then

$$p_{\text{unsafe}}(K) = \Pr[|e| > K] = 2(1 - \Phi(K/\sigma)). \quad (3.11)$$

Since fallback is triggered when the DT is unsafe and *does not* miss, the expected latency follows the same structure as Section 3.3 but with the tolerance-aware terms:

$$T_{\text{KID}} = t_{\text{ver}} + \frac{n_{\text{ver}}}{B} + p_{\text{unsafe}}(K)(1 - p_{\text{miss},K})\frac{n_{\text{data}}}{B}. \quad (3.12)$$

The empirical sweep in Chapter 5 instantiates this model and visualizes the safety-latency trade-off across K and n_{ver} .

3.5 Dynamic Evaluation Model: Random Walk with Drift

The static state model used in the parameter sweep (i.i.d. samples of S_t) is sufficient to estimate per-tick miss probabilities, but it cannot capture *error accumulation* over time. To study *continuous unsafe operation* and boundary breaches caused by missed detections, we introduce a time-correlated dynamic model where the robot drifts and the DT holds its last validated state.

We model the (scalar) robot state as a discrete-time random walk:

$$S_t = S_{t-1} + \delta_t, \quad (3.13)$$

where δ_t represents bounded motion noise (uniform increments) or unbounded disturbances (Gaussian increments). In particular, a uniform increment $\delta_t \sim \mathcal{U}(-\Delta_{step}, \Delta_{step})$ captures hard kinematic limits, while a Gaussian increment $\delta_t \sim \mathcal{N}(0, \sigma_{step}^2)$ captures occasional large excursions.

On the receiver side, the DT uses a ZOH predictor: in the absence of a correction event, it assumes the last synchronized state remains valid,

$$\hat{S}_t = \hat{S}_{t-1} \quad \text{unless a fallback correction is triggered.} \quad (3.14)$$

The instantaneous drift error is $E_t = |S_t - \hat{S}_t|$, which grows with time when left uncorrected.

At each tick, the protocol applies the K-ID tolerance gate:

$$\text{if } E_t \leq K : \text{no transmission (masked drift);} \quad \text{if } E_t > K : \text{verify and (usually) correct.} \quad (3.15)$$

When $E_t > K$, the Robot transmits a verifier and the DT attempts to detect inconsistency. With probability $p_{miss,K}$ a collision induces a *miss*: the DT accepts and does not correct, allowing drift to continue accumulating beyond K . Otherwise, the DT triggers fallback and resets $\hat{S}_t \leftarrow S_t$.

To keep the dynamic simulator consistent with the Random Oracle analysis in Section 3.4, we model misses at *verification events* (ticks with $E_t > K$) using the acceptance-set proxy

$$p_{miss,K}^{SHA} \approx \frac{|\mathcal{A}_K|}{2^{n_{ver}}}, \quad (3.16)$$

where $|\mathcal{A}_K|$ is computed over a quantized state domain (resolution Δ). For a 1D scalar state this yields $|\mathcal{A}_K| \approx 2\lfloor K/\Delta \rfloor + 1$.

Decoder compute cost. Beyond collision risk, the same acceptance-set size determines the DT-side compute and memory footprint: a direct implementation precomputes the verifier set $\mathcal{V}_K$ by hashing each quantized state in $\mathcal{A}_K$, i.e., $\mathcal{O}(|\mathcal{A}_K|)$ hash evaluations per DT update, and stores up to $|\mathcal{V}_K| \leq |\mathcal{A}_K|$ verifier codewords. This introduces a practical engineering trade-off between larger tolerances K (fewer fallbacks) and decoder workload.

Beyond per-tick rates, the drift model enables time-extended safety metrics. We report (i) the *breach duration* L_K , defined as the length of a consecutive streak where $E_t > K$, and (ii) a *physical danger* duration L_D for a fixed threshold D (application-specific). These quantities reveal how K creates a masking risk (drift grows without checks) and how collisions create an unbounded risk (checks occur but do not correct).

4 Implementation

This chapter summarizes the practical implementation used to instantiate and evaluate the theoretical models introduced in Chapter 3. The evaluation is organized around three complementary models. First, we implement the end-to-end ID latency model and generate bandwidth and desynchronization sensitivity plots. Second, we implement the tolerance-based K-ID protocol and perform a parameter sweep over (K, n_{ver}) to expose the safety-performance trade-off and identify Pareto-optimal operating points. Third, we implement a time-correlated dynamic model in which K-ID operates under random-walk drift, allowing us to study accumulation effects such as sustained boundary breaches after missed corrections.

4.1 Experimental Parameters

This chapter details the practical setup used to validate the theoretical models from Chapter 3. Unless otherwise stated, we use a baseline desynchronization probability of $p_{desync} = 0.1$; this value is held fixed for the bandwidth sweep and is varied explicitly when evaluating desynchronization sensitivity. We evaluate two data payloads: a small 96-bit message and a larger 4001-bit message. For each payload, we test verifier lengths of $n_{ver} \in \{4, 16\}$ bits, representing a small and a large verifier size.

4.1.1 Evaluation Scenarios (Bandwidth Baselines)

We use two bandwidth operating points, each tailored to the objective of the corresponding evaluation. For the baseline ID sensitivity results in Section 5.2 (Chapter 5), we report single-bandwidth slices at $B = 5$ Mbps to isolate the impact of prediction quality (p_{desync}) and payload size (n_{data}) without making the channel the sole bottleneck; this also complements the full bandwidth sweeps (10 kbps–10 Mbps). For the K-ID parameter sweep and Pareto analysis in Section 5.3, we adopt a bandwidth-constrained setting ($B = 1$ Mbps) because K-ID primarily trades computation for reduced traffic (fewer fallback transmissions), and this benefit is most visible when the transmission term n_{data}/B dominates.

In addition, we compute the theoretical collision probabilities (p_{miss}) for this parameter grid using the reliability bounds derived in Section 3.2. These values are used to instantiate the hybrid latency model consistently across the two encoding mechanisms.

4.1.2 LiDAR-Scale Scalability Parameters

To study how the compute overhead t_{ver} scales with high-dimensional sensor payloads, we perform an additional scalability experiment at $n_{ver} = 16$. We measure verifier generation time for payload sizes representative of LiDAR message chunks and frame-scale payloads:

$$n_{data} \in \{12 \text{ B}, 501 \text{ B}, 100 \text{ KB}, 1 \text{ MiB}, 5 \text{ MiB}\} \quad (4.1)$$

where 12 B and 501 B correspond to the 96-bit and 4001-bit baselines used elsewhere in the thesis. For [SHA-256](#), the payload is represented as a byte list of length $\lceil n_{data}/8 \rceil$. For [RS-ID](#), the payload is modeled as packed $\text{GF}(2^{n_{ver}})$ symbols with

$$k = \left\lceil \frac{n_{data}}{n_{ver}} \right\rceil \quad (4.2)$$

so that increasing payload size increases the number of field coefficients processed by the algebraic encoder.

All t_{ver} measurements report mean $\pm$ 95% CI half-width, and the timing configuration is chosen to keep the relative CI within approximately 5% for each point. The scripts used for these measurements are located in `experiments/`:

```
python3 experiments/measure_tver_ci_table.py \
  --nver-bits 16 \
  --ndata-bits 96 4001 819200 8388608 41943040 \
  --batches 60 --min-iters 200 \
  --out-csv experiments/results/tver_ci_table_nver16.csv

python3 experiments/plot_tver_ci_table.py \
  --csv experiments/results/tver_ci_table_nver16.csv \
  --out experiments/results/tver_ci_table_nver16.png
```

For the corresponding end-to-end latency visualization over payload size (reported in Chapter 5), we fix $B = 5$ Mbps and $p_{desync} = 0.1$ and evaluate the hybrid latency model at $n_{ver} = 16$ across the same payload sizes.

4.2 Data Packet Structure Abstraction

While the analysis in Chapter 3 models transmission cost in terms of n_{ver} and n_{data} , practical deployments encapsulate these fields in a communication frame. Figure 4.1 illustrates the abstract frame layout assumed in this thesis. In normal operation, the Robot transmits only a compact verifier; when verification fails, the protocol falls back to transmitting the full state payload. The [K-ID](#) protocol uses the same message structure as [ID](#); it differs only in the receiver-side acceptance rule (set membership rather than strict equality).

In the latency model, the header, timestamp, and checksum/CRC are treated as fixed overheads and are omitted for clarity. If required, these can be incorporated by adding a constant number of bits to both the verifier-only and fallback frames.

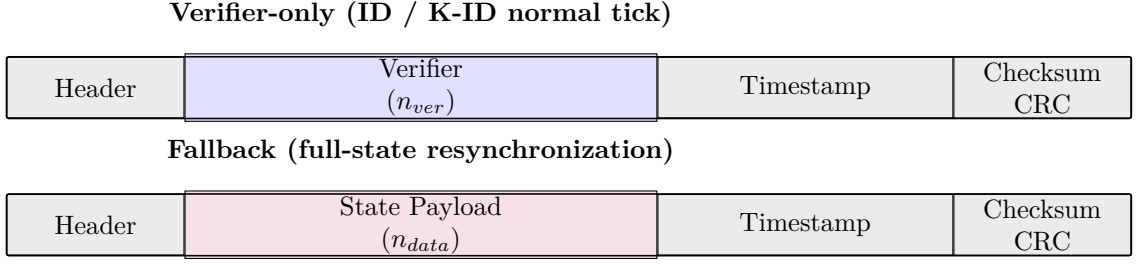


Figure 4.1: Abstract ID message format used for modeling. The normal path transmits only a verifier, while the fallback path transmits the full state payload. Fixed overhead fields are shown for completeness but are not explicitly modeled in the latency equations.

4.3 Software Architecture

This section summarizes the software structure used to keep the simulator modular.

4.3.1 Modular Simulator Architecture

Figure 4.2 shows the high-level decomposition used throughout the Python experiments. The simulator is structured around three interchangeable components—*Encoder*, *Channel/Dynamics*, and *Decoder/Decision*. This separation allows swapping SHA-256 and RS-ID encoders, switching between strict ID and tolerance-based K-ID decision rules, and evaluating both i.i.d. and time-correlated models without entangling implementation details.

The interface between Encoder and Channel is a transmitted *packet/frame*. In normal operation this contains only a verifier tag; on fallback it contains the full state payload. This matches the message abstraction in Figure 4.1.

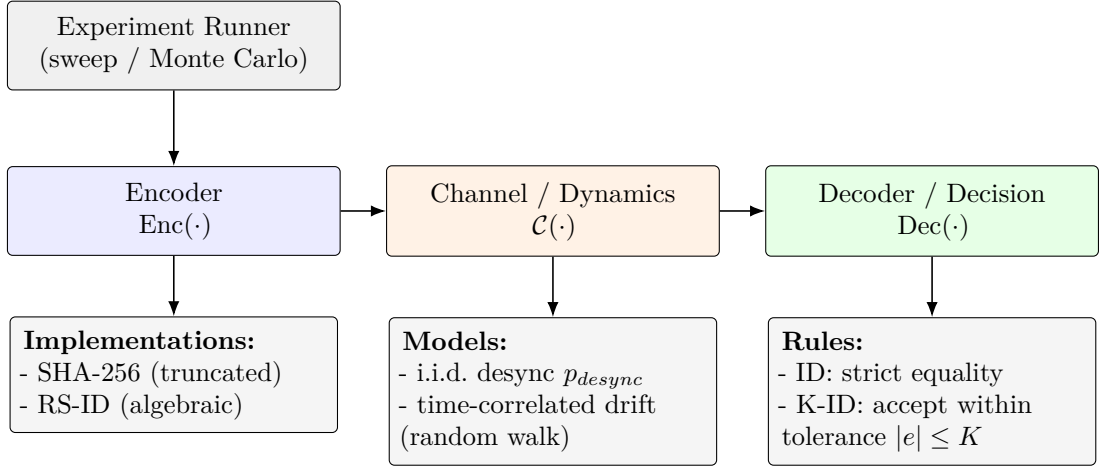


Figure 4.2: High-level simulator architecture used in the Python experiments. Encoder, Channel/Dynamics, and Decoder are decoupled so that coding schemes (SHA-256 vs. RS-ID), acceptance rules (ID vs. K-ID), and stochastic models (i.i.d. vs. time-correlated drift) can be varied independently.

4.4 Latency Plot Generation

To quantify the implications of the hybrid latency model, we evaluate the expected latency under systematic sweeps of the channel bandwidth B and the desynchronization probability p_{desync} . All plots are generated for multiple payload sizes n_{data} , verifier lengths n_{ver} , and for both encoding mechanisms (SHA-256 vs. RS-ID), allowing us to compare regimes where ID provides a net latency advantage over traditional transmission.

The latency evaluation uses the parameter grid defined in Section 4.1. The compute overheads t_{ver} are taken from empirical measurements (Table 5.1), and the collision probabilities p_{miss} are computed from the theoretical bounds (Table 5.2).

4.4.1 Logic

For each configuration (n_{data}, n_{ver}) and each system (SHA-256 vs. RS-ID), the script evaluates the expected latency

$$L_{ID}(B) = t_{ver} + \frac{n_{ver}}{B} + p_{desync} (1 - p_{miss}) \frac{n_{data}}{B} \quad (4.3)$$

over a log-spaced bandwidth grid $B \in [B_{min}, B_{max}]$. The traditional baseline is computed as $L_{trad}(B) = n_{data}/B$. The resulting curves are plotted on log-log axes to reveal the break-even regime where ID-based synchronization becomes beneficial.

The generated figures are shown in Chapter 5 for $n_{data} = 96$ bits (Figure 5.1a) and for $n_{data} = 4001$ bits (Figure 5.1b).

In addition, we visualize the sensitivity to desynchronization by sweeping $p_{desync} \in [0, 1]$ at a fixed bandwidth and plotting L_{ID} against p_{desync} for the same parameter grid (Figures 5.2a and 5.2b).

4.4.2 SHA-256

The collision probability for a truncated hash function under the Random Oracle Model is independent of the data size and is given by Equation 3.4:

$$P_{miss}^{SHA} = 2^{-n_{ver}} \quad (4.4)$$

The corresponding numerical values for our selected verifier lengths are reported in Table 5.2 in Chapter 5.

4.4.3 RS-ID

The collision probability for the algebraic construction is bounded by the Schwartz-Zippel Lemma, as shown in Equation 3.2. This bound depends on both data size (n_{data}) and verifier length (n_{ver}).

$$P_{miss}^{RS} \leq \frac{\lceil n_{data,bits}/n_{ver} \rceil}{2^{n_{ver}}} \quad (4.5)$$

The computed values for $n_{data} \in \{96, 4001\}$ bits and $n_{ver} \in \{4, 16\}$ bits (with the bound capped at 1 when applicable) are reported in Table 5.2 in Chapter 5. These values highlight how the theoretical safety of RS-ID degrades with larger data sizes for a fixed verifier length, a key distinction from the cryptographic approach.

4.5 K-ID Parameter Sweep Setup

In addition to the bit-based latency evaluation, we implement the tolerance-based K-ID model introduced in Section 3.4 and perform a parameter sweep over (K, n_{ver}) to quantify the safety-latency trade-off of the K-ID decision rule.

4.5.1 Model and Metrics

The K-ID sweep uses a truncated SHA-256 verifier and reports the conditional miss rate $\Pr[\text{accept} \mid \text{unsafe}]$ (Equation (3.10)), the fallback rate $\Pr[\text{fallback}]$ across ticks, the expected latency from the hybrid model (Equation (3.12)), and the speedup defined as the ratio between the traditional transmission time (n_{data}/B) and the expected K-ID latency. For the Pareto analysis in Chapter 5, we additionally use the empirical *overall risk* $\Pr[\text{unsafe} \cap \text{accept}]$, estimated as `miss_count/ticks`, to combine event frequency with conditional acceptance.

4.5.2 State Space and Quantization

To avoid an artificially small discrete domain (which can force the empirical miss rate to become exactly zero for moderate n_{ver}), the robot state is modeled as a continuous random variable and then quantized: We sample $S_t \sim \mathcal{N}(\mu, \sigma^2)$, clip it to a physical range $[S_{min}, S_{max}]$, quantize it to a resolution Δ (default $\Delta = 0.1$), and hash the resulting integer index. This yields a discrete domain size of approximately $(S_{max} - S_{min})/\Delta + 1$.

4.5.3 Parameter Grid

Unless otherwise stated, we use a LiDAR-scale payload ($n_{data} = 4001$ bits) at $B = 1$ Mbps with a compute overhead of $t_{ver} = 2.13 \mu s$ (SHA-256, 96-bit state; Table 5.1). The robot state is clipped to $[0, 100]$ and quantized with $\Delta = 0.1$. We evaluate two prediction-error models: (i) a Gaussian model $S_t \sim \mathcal{N}(\mu, \sigma^2)$ with $\mu = 50$ and $\sigma = 12$, and (ii) a variance-matched Uniform model $S_t \sim \mathcal{U}[\mu - a, \mu + a]$ with $a = \sqrt{3}\sigma$ to ensure comparable variance. We sweep $n_{ver} \in \{4, 8, 12, 16\}$ and $K \in \{2, 5, 10, 20\}$, using 200,000 Monte Carlo ticks per configuration with a fixed seed of 42.

4.5.4 Reproducibility

The sweep and plot generation are implemented in the `experiments/` folder: The simulation is implemented in `experiments/kid_parameter_sweep.py` (optionally writing a CSV), and the plotting script `experiments/plot_kid_parameter_sweep.py` reads that CSV and produces the figures used in Chapter 5. The following commands reproduce the CSV and plots (paths are relative to the repository root):

```
python3 experiments/kid_parameter_sweep.py \
  --n-data-bits 4001 --bandwidth 1e6 --ticks 200000 --seed 42 \
  --robot-mean 50 --robot-std 12 --physical-min 0 --physical-max 100 \
  --value-resolution 0.1 --t-compute-us 2.13 \
  --nver 4 8 12 16 --k 2 5 10 20 \
  --system both --error-dist normal \
  --out-csv experiments/results/kid_sweep_gaussian_both.csv

python3 experiments/plot_kid_parameter_sweep.py \
  --csv experiments/results/kid_sweep_gaussian_both.csv \
  --out-dir experiments/results --prefix kid_sweep_gaussian_both

python3 experiments/kid_parameter_sweep.py \
  --n-data-bits 4001 --bandwidth 1e6 --ticks 200000 --seed 42 \
  --robot-mean 50 --robot-std 12 --physical-min 0 --physical-max 100 \
  --value-resolution 0.1 --t-compute-us 2.13 \
  --nver 4 8 12 16 --k 2 5 10 20 \
  --system both --error-dist uniform \
  --out-csv experiments/results/kid_sweep_uniform_both.csv

python3 experiments/plot_kid_parameter_sweep.py \
  --csv experiments/results/kid_sweep_uniform_both.csv \
  --out-dir experiments/results --prefix kid_sweep_uniform_both
```

4.6 Dynamic Drift Experiment (Random Walk & Correction)

To evaluate time-correlated error accumulation (Section 3.5), we implement a dynamic simulator in which the robot state evolves as a random walk and the DT uses a ZOH predictor. The protocol applies the K-ID tolerance gate each tick and only attempts verification when the

drift exceeds K . A miss (collision) during verification prevents a correction and allows the drift to accumulate further, producing extended boundary breaches.

4.6.1 Parameterization

Unless otherwise stated, the reported dynamic figures use a Gaussian step model $\delta_t \sim \mathcal{N}(0, \sigma_{step}^2)$ with $\sigma_{step} = 2$ and a quantization resolution of $\Delta = 1$ for the acceptance-set size proxy $|\mathcal{A}_K| \approx 2\lfloor K/\Delta \rfloor + 1$. We sweep $K \in \{2, 5, 10, 20\}$ and verifier sizes $n_{ver} \in \{8, 12, 16\}$ over 120,000 ticks per run with 10 replicates (base seed 42). For the physical danger metric, we set $D = 15$ to visualize sustained unsafe operation within the simulated time horizon.

4.6.2 Reproducibility

The sweep and plot generation are implemented in the `experiments/` folder. The simulation writes a CSV, and the plotting script generates the thesis-ready figures in `thesis_report/figures/plots/`:

```
python3 experiments/random_walk_drift_sweep.py \
  --ticks 120000 --replicates 10 --seed 42 \
  --step-dist gaussian --step-param 2.0 \
  --value-resolution 1.0 --danger-threshold 15 \
  --k 2 5 10 20 --nver 8 12 16 \
  --out-csv experiments/results/random_walk_drift_sweep_gaussian_step2_D15.csv

python3 experiments/plot_random_walk_drift_sweep.py \
  --csv experiments/results/random_walk_drift_sweep_gaussian_step2_D15.csv \
  --out-dir thesis_report/figures/plots \
  --name-suffix step2_D15 \
  --demo --demo-ticks 2500 --demo-seed 125 \
  --demo-step-dist gaussian --demo-step-param 2.0 \
  --demo-k 10 --demo-danger 15 --demo-nver-a 16 --demo-nver-b 8
```

To quantify the decoder-side compute footprint of K-ID and to summarize the full (K, n_{ver}) grid, we generate additional derived plots from the static sweep CSVs:

```
python3 experiments/plot_kid_decoder_cost.py \
  --csv experiments/results/kid_sweep_gaussian_both_ndata4001.csv \
  --system sha256_trunc \
  --out thesis_report/figures/plots/kid_decoder_cost_gaussian_sha256.png

python3 experiments/plot_kid_heatmaps.py \
  --csv experiments/results/kid_sweep_gaussian_both_ndata4001.csv \
  --system sha256_trunc \
  --title "Gaussian (n_data=4001)" \
  --out thesis_report/figures/plots/kid_heatmap_gaussian_sha256.png

python3 experiments/plot_kid_heatmaps.py \
  --csv experiments/results/kid_sweep_uniform_both_ndata4001.csv \
```

```
--system sha256_trunc \  
--title "Uniform (n_data=4001)" \  
--out thesis_report/figures/plots/kid_heatmap_uniform_sha256.png  
  
python3 experiments/plot_kid_resolution_sensitivity.py \  
--system sha256_trunc --error gaussian --robot-std 12 \  
--nver 8 12 16 --k 2 5 10 20 --delta 1 0.1 0.01 \  
--out thesis_report/figures/plots/kid_delta_sensitivity_gaussian_sha256.png
```

For a survival-style view of transmission intervals in the dynamic drift model, we export inter-transmission times (IFT) and plot empirical CDFs:

```
python3 experiments/random_walk_drift_sweep.py \  
--ticks 120000 --replicates 10 --seed 42 \  
--step-dist gaussian --step-param 2.0 \  
--value-resolution 1.0 --danger-threshold 15 \  
--k 2 5 10 20 --nver 8 12 16 \  
--out-csv experiments/results/random_walk_drift_sweep_gaussian_step2_D15.csv \  
--out-ift-csv experiments/results/random_walk_ift_gaussian_step2_D15.csv  
  
python3 experiments/plot_random_walk_ift_cdf.py \  
--mode by_nver \  
--ift-csv experiments/results/random_walk_ift_gaussian_step2_D15_K*_nver*.csv \  
--out thesis_report/figures/plots/random_walk_drift_gaussian_step2_D15_ift_cdf.png
```

5 Results and Evaluation

This chapter presents the empirical evaluation of the proposed [ID](#)-based synchronization protocols. We first characterize the system’s fundamental parameters—computational overhead and collision probability—to calibrate the theoretical models. We then perform a sensitivity analysis to identify the bandwidth and prediction regimes where [ID](#) offers a latency advantage over traditional transmission. Next, we conduct a comprehensive parameter sweep of the [K-ID](#) protocol, quantifying the safety-performance trade-offs and identifying Pareto-optimal configurations for deployment. We conclude with a dynamic evaluation based on the random walk drift model to highlight time-correlated risk accumulation.

5.1 Empirical Parameter Verification

To instantiate the latency models derived in Chapter [3](#), we first experimentally verify the computational cost (t_{ver}) and the theoretical reliability limits (p_{miss}) of the encoding mechanisms.

5.1.1 Encoding Latency (t_{ver})

The encoding time t_{ver} sets the irreducible latency floor for [ID](#), even when bandwidth is effectively infinite. We measured t_{ver} over 100,000 operations for both [SHA-256](#) and [RS-ID](#). The results, summarized in Table [5.1](#), reveal a distinct structural divergence between the two approaches.

Table 5.1: Measured verification time t_{ver} (mean $\pm$ 95% CI half-width). **SHA-256** shows stable performance for fixed payloads, while **RS-ID** is sensitive to the verifier length due to field arithmetic complexity.

System (Payload)	Verifier Length (n_{ver})	
	4 bits	16 bits
<i>Cryptographic (SHA-256)</i>		
96 bits	$2.13 \pm 0.04 \mu s$	$2.24 \pm 0.01 \mu s$
4001 bits	$14.65 \pm 0.31 \mu s$	$15.41 \pm 0.09 \mu s$
<i>Algebraic (RS-ID)</i>		
96 bits	$14.55 \pm 0.39 \mu s$	$3.79 \pm 0.02 \mu s$
4001 bits	$51.78 \pm 0.94 \mu s$	$38.57 \pm 0.36 \mu s$

SHA-256 demonstrates robust stability; its runtime is dominated by the input block size and is effectively invariant to the output truncation length (n_{ver}). In contrast, **RS-ID** displays a counter-intuitive behavior where *increasing* the verifier length *decreases* latency (e.g., $14.55 \rightarrow 3.79 \mu s$). This is an artifact of the polynomial construction: a larger n_{ver} allows more bits to be packed into each field coefficient, reducing the total number of coefficients (k) required to represent the payload.

5.1.2 Theoretical Reliability Limits

Table 5.2 presents the theoretical collision probabilities calculated from the bounds in Chapter 3.

Table 5.2: Theoretical collision probability p_{miss} . **RS-ID** fails at short blocklengths ($n_{ver} = 4$) due to field saturation ($p_{miss} = 1.0$), whereas **SHA-256** maintains robust collision resistance.

System	Verifier Length (n_{ver})	
	4 bits	16 bits
SHA-256 (Random Oracle)	6.25×10^{-2}	1.53×10^{-5}
RS-ID (Schwartz-Zippel)	1.00 (<i>Saturated</i>)	9.16×10^{-5}

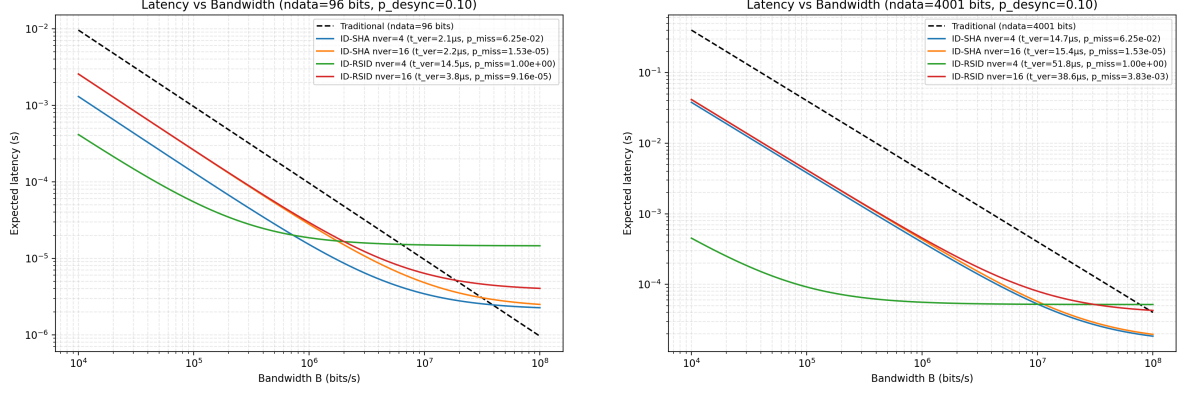
One critical finding is the failure of **RS-ID** at $n_{ver} = 4$. With a field size of only $q = 16$, the polynomial degree d for a 96-bit payload exceeds the field size ($d \geq q$), and the bound saturates at $p_{miss} = 1.0$. This confirms that algebraic codes require a minimum field headroom to function, establishing a hard lower bound on n_{ver} that does not exist for cryptographic hashes.

5.2 End-to-End Latency Evaluation

We now integrate the measured parameters into the hybrid latency model to evaluate system performance under varying network constraints.

5.2.1 The Break-Even Analysis (Bandwidth Sensitivity)

Figure 5.1 analyzes the expected latency across a bandwidth spectrum from 10 kbps to 10 Mbps.



(a) Small payload (96 bits): cross-over at ≈ 2 Mbps. (b) Large payload (4001 bits): ID advantage persists > 10 Mbps.

Figure 5.1: End-to-end latency vs. channel bandwidth. The plots reveal the break-even point where the computational cost of ID (t_{ver}) intersects with the transmission cost of the baseline. In the low-bandwidth regime ($B < 1$ Mbps), ID offers up to $10\times$ latency reduction.

The results illustrate the fundamental trade-off between computation and communication. In the transmission-bound regime ($B < 1$ Mbps), latency is dominated by the physical transmission time (T_{trans}), and ID outperforms the baseline by compressing the required transmission from n_{data} to n_{ver} . In the computation-bound regime ($B > 10$ Mbps), transmission becomes negligible and the latency floor is set by the encoding time t_{ver} ; since the traditional scheme has zero encoding overhead, it eventually becomes faster. For large payloads (Figure 5.1b), the break-even point shifts to higher bandwidths, indicating that ID-based synchronization becomes increasingly valuable as data complexity grows.

5.2.2 Operational Bounds (Prediction Sensitivity)

Figure 5.2 examines the impact of the DT's prediction quality (p_{desync}) at a fixed bandwidth of 5 Mbps. The intersection points in Figure 5.2 define the viability region of the protocol. For standard telemetry (96 bits), ID requires a prediction accuracy better than 20% ($p_{desync} < 0.8$) to yield net benefits. This indicates that ID remains practical in dynamic environments, provided the DT's kinematic model remains sufficiently coupled to reality.

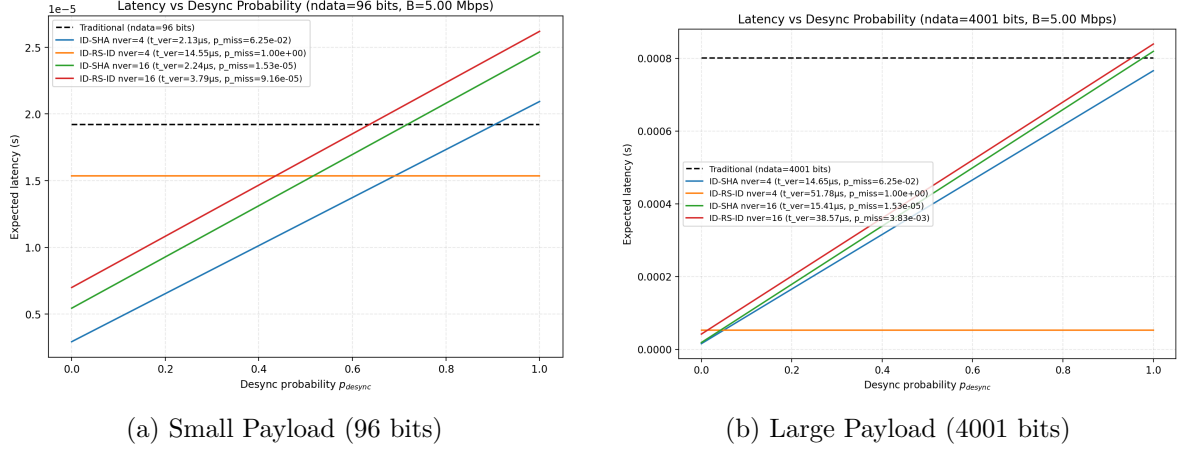


Figure 5.2: Latency sensitivity to prediction error. **ID** relies on accurate predictions. For small payloads (a), the system must maintain $p_{desync} < 0.8$ to outperform the baseline. For large payloads (b), the transmission savings are so significant that **ID** remains superior even with poor predictions.

5.2.3 LiDAR-Scale Payload Scalability

The baseline measurements above use small telemetry payloads (96 bits and 4001 bits). For high-dimensional sensors such as **Light Detection and Ranging (LiDAR)**, the payload size can grow by orders of magnitude. We therefore extend the t_{ver} benchmark to payloads up to 5 MiB (Section 4.1.2).

Figure 5.3 (visualized from Table 5.3) highlights a clear crossover in computational efficiency governed by payload size. For **small payloads** (< 100 B), **RS-ID** exhibits a slight performance edge (e.g., $1.81 \mu s$ vs. $2.06 \mu s$ at 96 bits), consistent with a small fixed overhead for polynomial evaluation while **SHA-256** incurs a startup cost due to message padding and the fixed compression routine. For **large payloads** (> 1 KiB), **SHA-256** significantly outperforms **RS-ID** (e.g., about $2\times$ faster at 5 MiB). While both approaches scale linearly in the number of processed bytes ($O(N)$), **SHA-256** relies primarily on bitwise operations that are heavily optimized on modern CPUs, whereas the current **RS-ID** implementation is dominated by memory-heavy finite-field multiplication table lookups.

Overall, for high-throughput sensor payloads (**LiDAR**/video), cryptographic hashing provides superior scalability in this prototype implementation. For distinct control signals and small telemetry payloads, both methods remain computationally negligible (on the order of a few microseconds).

Table 5.3: Measured verifier generation time t_{ver} for LiDAR-scale payload sizes ($n_{ver} = 16$). Values are mean $\pm$ 95% CI half-width; CI is controlled to $\leq 5\%$ relative error.

Payload	n_{data} (bits)	SHA-256 t_{ver} (μs)	RS-ID t_{ver} (μs)
12 B	96	2.06 ± 0.06	1.81 ± 0.02
501 B	4001	15.27 ± 0.30	24.69 ± 0.13
100 KB	819200	2702.45 ± 20.69	5208.16 ± 139.68
1 MiB	8388608	28388.94 ± 916.15	52728.32 ± 2396.59
5 MiB	41943040	140418.57 ± 1286.59	271339.25 ± 7741.95

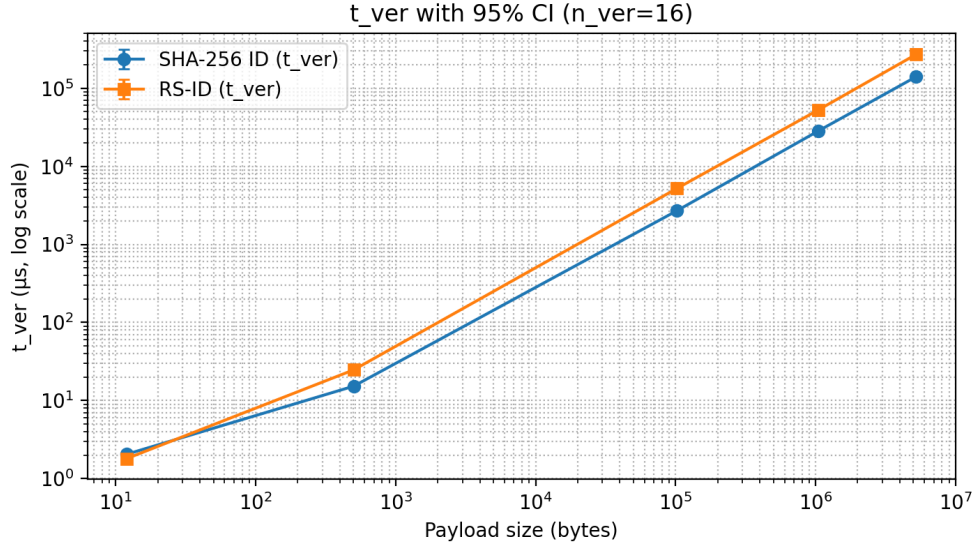


Figure 5.3: t_{ver} scalability vs. payload size at $n_{ver} = 16$ (log-log). Error bars show 95% CI half-width with relative CI controlled to $\leq 5\%$.

5.2.4 Overall Latency vs. Payload Size (Fixed n_{ver})

To connect the scalability measurement to end-to-end performance, we evaluate the hybrid latency model at a fixed verifier size ($n_{ver} = 16$), fixed bandwidth ($B = 5$ Mbps), and $p_{desync} = 0.1$, while varying the payload size from telemetry-scale to LiDAR-scale. Figure 5.4 reports the resulting expected latency.

As the compression ratio increases (fixed n_{ver} while n_{data} grows), the ordering between the two ID variants becomes consistent with the t_{ver} scalability results in Figure 5.3: the SHA-256-based scheme achieves lower end-to-end latency than RS-ID. Intuitively, at fixed n_{ver} and fixed p_{desync} , both schemes share essentially the same expected communication component (the verifier transmission on non-fallback ticks, and the same fallback transmission model); therefore, the gap between the curves in Figure 5.4 is dominated by the compute floor t_{ver} . Since SHA-256 scales with a lower per-byte cost than the current RS-ID implementation, it remains faster as n_{data} increases.

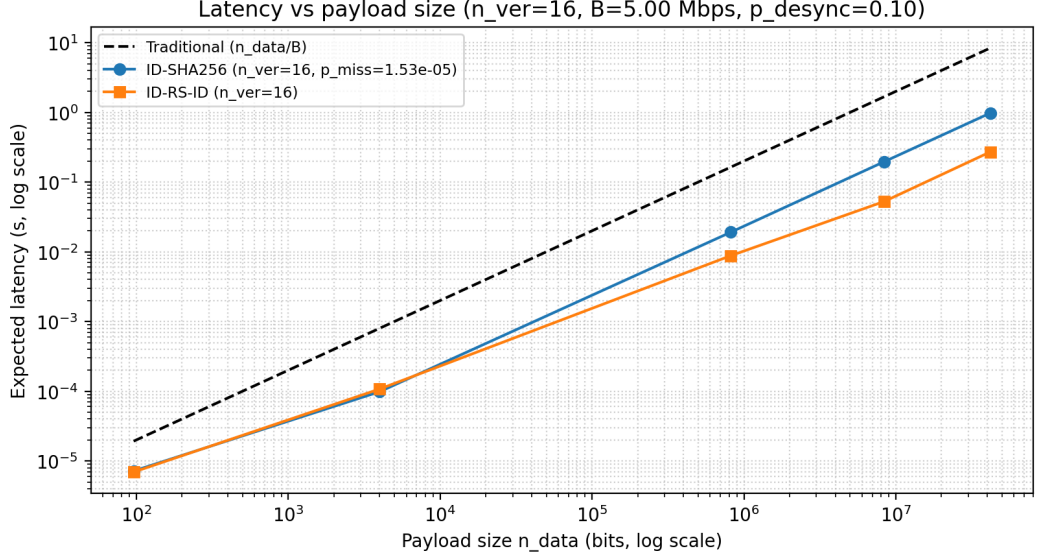


Figure 5.4: Expected latency vs. payload size ($n_{ver} = 16$, $B = 5$ Mbps, $p_{desync} = 0.1$). The ID-based schemes incur a compute floor (t_{ver}) but can reduce communication cost to n_{ver}/B on non-fallback ticks.

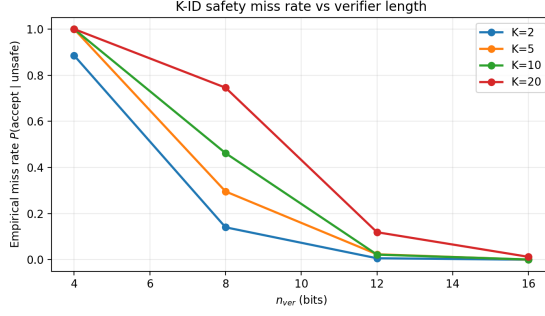
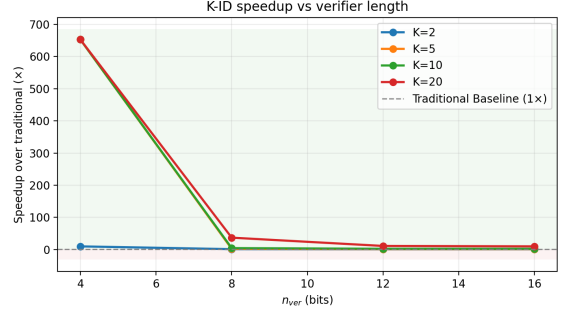
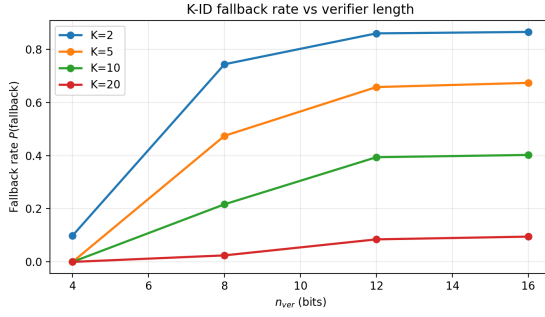
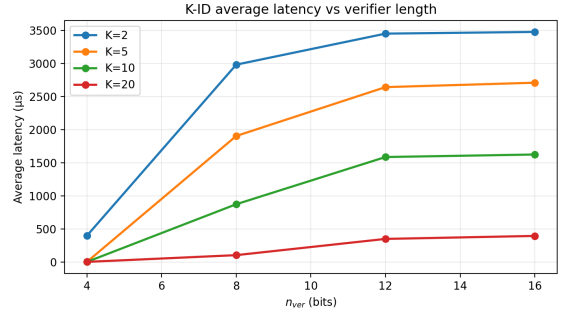
5.3 K-ID: Quantifying the Safety-Performance Trade-off

Having established the baseline performance, we now evaluate the **K-ID** extension, which introduces semantic tolerance (K) to filter benign deviations. We sweep $K \in \{2, 5, 10, 20\}$ and $n_{ver} \in \{4, 8, 12, 16\}$ to map the design space.

5.3.1 Mechanism Analysis

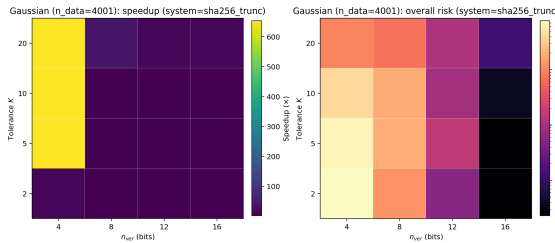
Figure 5.5 dissects how the protocol parameters influence the four key system metrics under a Gaussian error model. Figure 5.9 reports the same metrics under a variance-matched Uniform error model.

Grid-level summary and hidden decoder cost. While the line plots emphasize one parameter at a time, a compact view of the full (K, n_{ver}) design grid is useful for selecting practical operating regions. Figure 5.6 summarizes speedup and overall risk over the complete grid. In addition, K-ID introduces a DT-side cost that is not visible in bandwidth/latency figures: the verifier set $\mathcal{V}_K$ is built by hashing all acceptable quantized states in $\mathcal{A}_K$, so decoder work scales with $|\mathcal{A}_K| \approx 2\lfloor K/\Delta \rfloor + 1$. Figure 5.7 visualizes this growth.

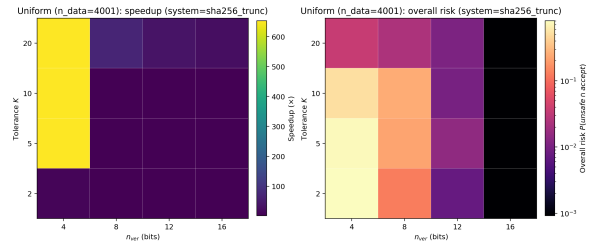

 (a) Conditional miss rate: increases with K and is suppressed exponentially with n_{ver} .

 (b) Speedup factor: driven by K . Larger tolerance reduces fallbacks, boosting speedup.

 (c) Fallback rate: higher n_{ver} increases fallback by detecting more unsafe states.


(d) Expected latency: inverse of speedup. Minimizing fallback is key to low latency.

Figure 5.5: Anatomy of the K-ID trade-off (Gaussian error model). Parameters: $n_{data} = 4001$ bits, $B = 1$ Mbps, $t_{ver} = 2.13 \mu s$, 200,000 ticks per configuration, $K \in \{2, 5, 10, 20\}$, and $n_{ver} \in \{4, 8, 12, 16\}$. The results demonstrate the conflicting roles of tolerance (K) and verification stringency (n_{ver}). Increasing K improves performance (b, d) by ignoring minor errors, but raises collision risk (a). Increasing n_{ver} restores safety but adds transmission overhead.



(a) Gaussian model (SHA-256 truncation).



(b) Variance-matched Uniform model (SHA-256 truncation).

Figure 5.6: Heatmap summary of the K-ID grid over (K, n_{ver}) . The left panel in each figure shows speedup; the right panel shows empirical overall risk $P(\text{unsafe} \cap \text{accept})$ (log-scaled).

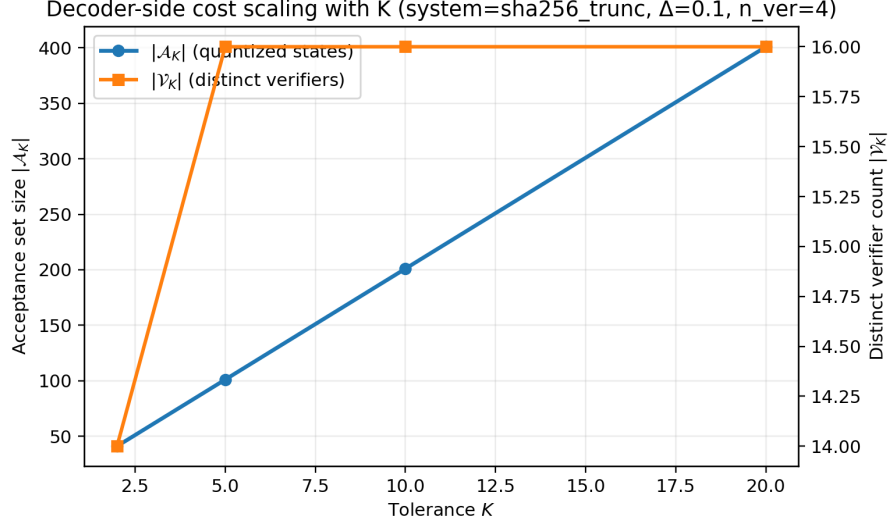


Figure 5.7: Hidden DT-side cost of K-ID: acceptance-set size $|\mathcal{A}_K|$ and distinct verifier count $|\mathcal{V}_K|$ grow with tolerance K (here for $\Delta = 0.1$). This motivates treating K as a joint network-and-compute design parameter.

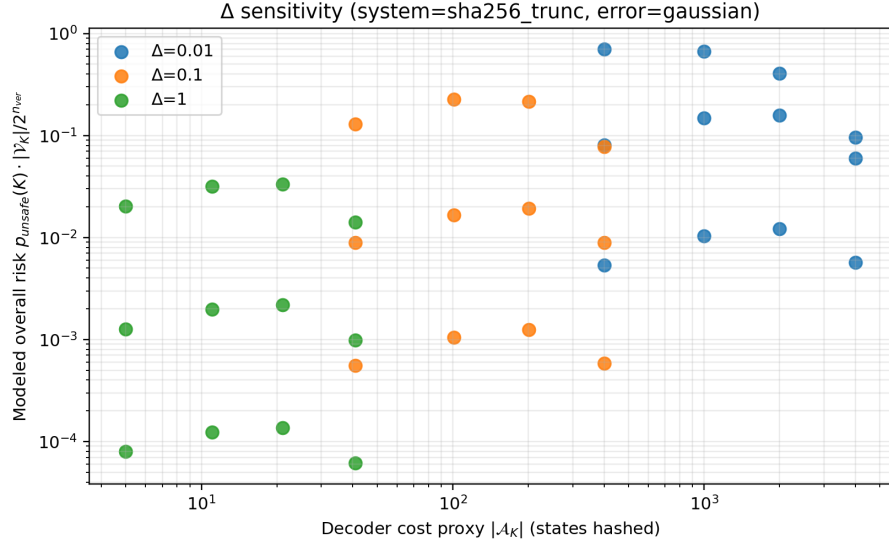
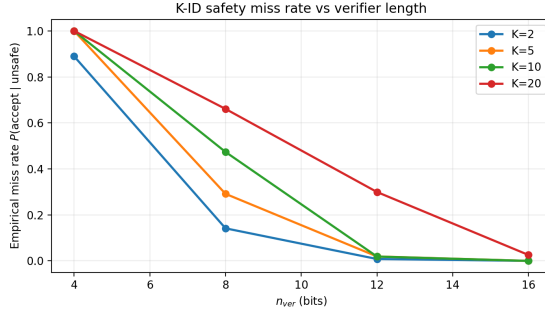
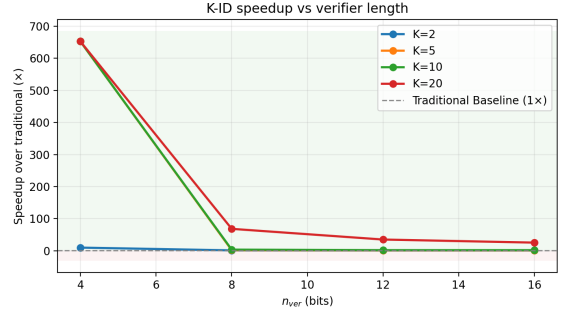


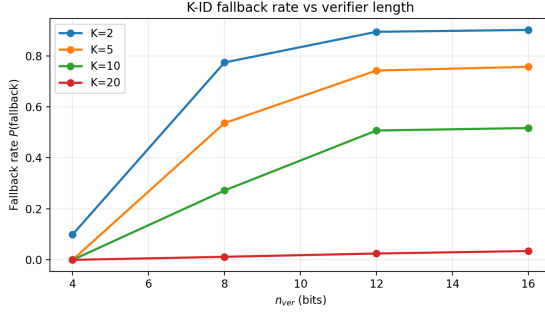
Figure 5.8: Quantization-resolution sensitivity (Δ). A finer Δ increases decoder cost $|\mathcal{A}_K|$ (more hashes) and can shift the modeled overall-risk proxy. This highlights that Δ should be selected based on sensor precision and DT compute budget rather than treated as an arbitrary constant.



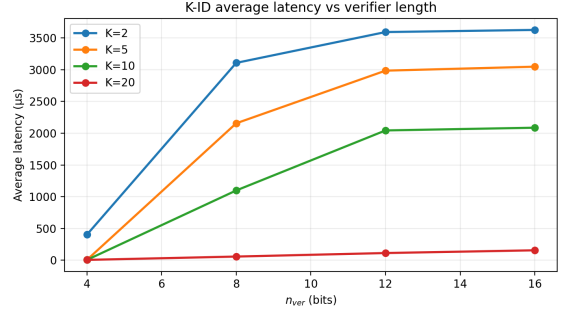
(a) Conditional miss rate: increases with K and is suppressed exponentially with n_{ver} .



(b) Speedup factor: driven by K . Larger tolerance reduces fallbacks, boosting speedup.



(c) Fallback rate: higher n_{ver} increases fallback by detecting more unsafe states.



(d) Expected latency: inverse of speedup. Minimizing fallback is key to low latency.

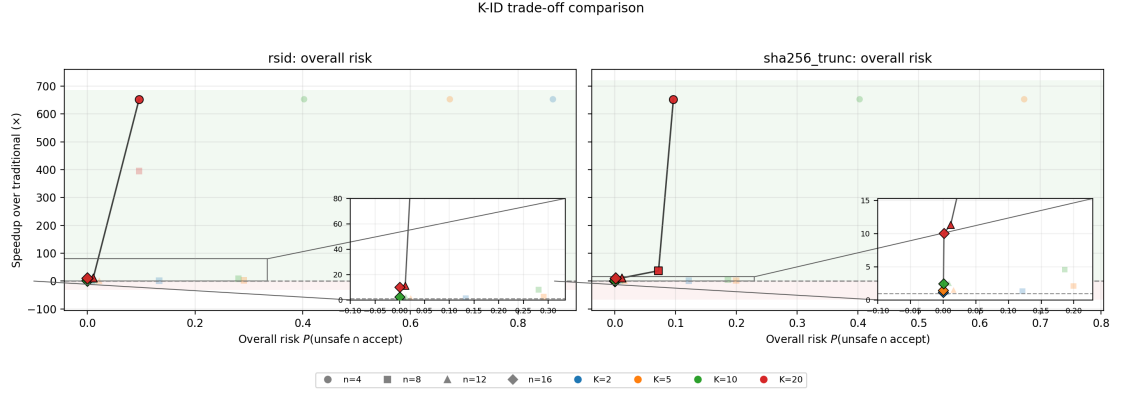
Figure 5.9: Anatomy of the **K-ID** trade-off (Uniform error model, variance-matched). Parameters: $n_{data} = 4001$ bits, $B = 1$ Mbps, $t_{ver} = 2.13 \mu s$, 200,000 ticks per configuration, $K \in \{2, 5, 10, 20\}$, and $n_{ver} \in \{4, 8, 12, 16\}$. The Uniform distribution is variance-matched to the Gaussian model.

Increasing the tolerance K creates a semantic filter. As seen in Fig. 5.5(c), moving from $K = 2$ to $K = 20$ drastically cuts the fallback rate, because the system stops correcting minor sensor noise. This drives the speedup in Fig. 5.5(b). The trade-off is that a larger acceptance set $\mathcal{A}_K$ increases the chance of accepting an unsafe state through a collision proxy (Fig. 5.5(a)).

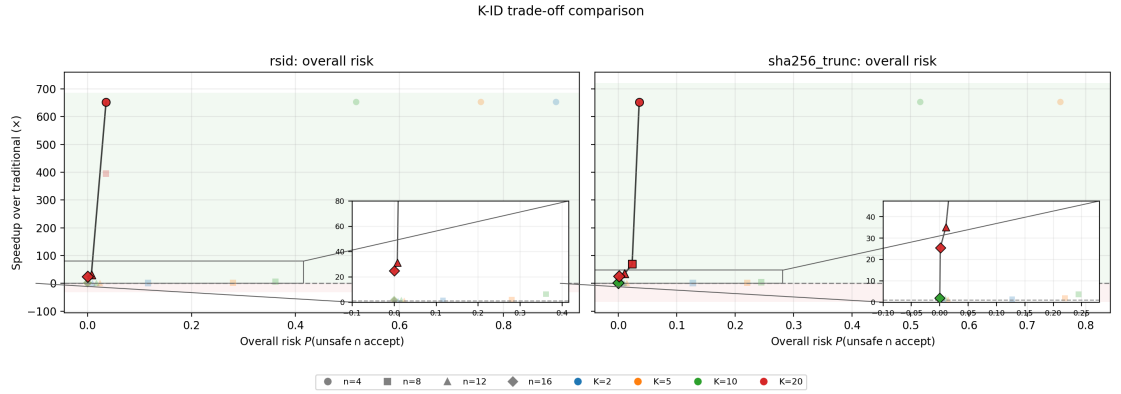
The verifier length n_{ver} is the primary safety knob. It suppresses the conditional miss rate exponentially (linear log-plot in Fig. 5.5(a)), counteracting the risk introduced by large K . In practice, high-tolerance **K-ID** configurations therefore require longer verifiers to maintain safety.

5.3.2 Pareto Optimization for Deployment

To guide practical deployment, we map the configurations onto a Speedup vs. Risk plane. We show results for two prediction-error models: a Gaussian error model (Normal) and a variance-matched Uniform error model. Unless otherwise stated, the Pareto results use $n_{data} = 4001$ bits at $B = 1$ Mbps. Figure 5.10 visualizes the corresponding Pareto frontiers.



(a) Gaussian error model



(b) Uniform error model (variance-matched)

Figure 5.10: Pareto frontier of K-ID design under two error models. The x-axis is the empirical *overall risk* $P(\text{unsafe} \cap \text{accept})$ per tick (i.e., miss events per tick), and the y-axis is speedup over traditional transmission. The solid line connects Pareto-nondominated configurations. Insets zoom into the low-risk region where points would otherwise overlap; the corresponding Pareto-optimal (K, n_{ver}) settings are listed in Table 5.4.

Figure 5.10 highlights how the assumed noise model changes the attainable safety–performance trade-off. Under the variance-matched Uniform model (Fig. 5.10b), the distribution is bounded; once the tolerance K exceeds the maximum deviation supported by the model, unsafe events effectively disappear ($P(\text{unsafe}) \approx 0$), producing a near-vertical rise close to zero overall risk. This captures the practical case of sensors with known precision limits, where K-ID can be tuned to reduce traffic without materially compromising safety.

Under the Gaussian model (Fig. 5.10a), the distribution has unbounded support, so there remains a non-zero tail probability of large deviations for any finite K . As a result, the frontier forms a smooth convex curve: higher speedups require accepting a gradually increasing (but quantifiable) residual risk. In this setting, selecting (K, n_{ver}) becomes a risk-budget decision.

Across both models, K sets the performance scale because it controls how often the protocol falls back, while n_{ver} primarily shifts operating points leftward by reducing collision probability. Table 5.4 lists the Pareto-nondominated configurations for each encoder.

There is no single best parameter set; the optimal choice depends on the application’s risk budget. For critical tasks (e.g., human-robot interaction), designers should select points on the far left of Figure 5.10 (e.g., $K = 2, n_{\text{ver}} = 16$), accepting lower speedups ($\approx 1.5\times$) for minimal risk ($< 10^{-11}$). For monitoring tasks (e.g., logging), relaxing constraints toward the right (e.g., $K = 10, n_{\text{ver}} = 12$) yields significant bandwidth savings ($> 4\times$) with manageable risk. Overall, the frontier shows that K-ID enables a controlled and quantifiable trade-off between safety margin and network performance.

Table 5.4: Pareto-nondominated **K-ID** configurations under Gaussian and variance-matched Uniform models ($n_{data} = 4001$ bits, $B = 1$ Mbps). Overall risk is empirical $P(\text{unsafe} \cap \text{accept})$ per tick.

Model	System	K	n_{ver}	Overall risk	Speedup ($\times$)
Gaussian	rsid	10	16	1.5e-04	2.46
		20	16	1.8e-04	9.95
		20	12	0.0111	11.28
		20	4	0.0962	652.69
	sha256_trunc	2	16	0	1.15
		5	16	2.3e-04	1.48
		10	16	4.0e-04	2.46
		20	16	0.0012	10.05
		20	12	0.0115	11.33
		20	8	0.0717	37.06
		20	4	0.0962	652.69
Uniform	rsid	20	16	0	24.89
		20	12	0.0074	31.44
		20	4	0.0356	652.69
	sha256_trunc	10	16	0	1.92
		20	16	9.2e-04	25.47
		20	12	0.0107	35.09
		20	8	0.0236	68.39
		20	4	0.0356	652.69

5.4 Dynamic Evaluation: Random Walk with Drift

The parameter sweep in Section 5.3 estimates per-tick risk under an i.i.d. state model. However, safety-critical systems are impacted by *time correlation*: a missed correction can persist and compound as the robot continues moving. We therefore evaluate a dynamic drift model (Section 3.5) in which the robot evolves as a random walk and the **DT** uses a **ZOH** predictor.

5.4.1 Qualitative Behavior (Drift & Escape Time Series)

Figure 5.11 visualizes the *drift and escape* process directly in the state domain. The blue curve shows the robot true state S_t undergoing a random walk, while the orange dashed curve shows the **DT** estimate $\hat{S}_t$ under a **ZOH**-style reference (held constant). The green band highlights the semantic safety set $\mathcal{A}_K$ and the red background indicates the unsafe region.

The green markers indicate successful detections: when the trajectory breaches the boundary K , the protocol triggers fallback and the system is corrected (visualized here as an immediate reset to the reference). The red crosses are the critical events: a hash collision causes a miss exactly at the boundary breach, so the **DT** fails to react and the trajectory continues to drift deeper into the red region. This makes the *accumulating, potentially unbounded* risk visible in a single glance.

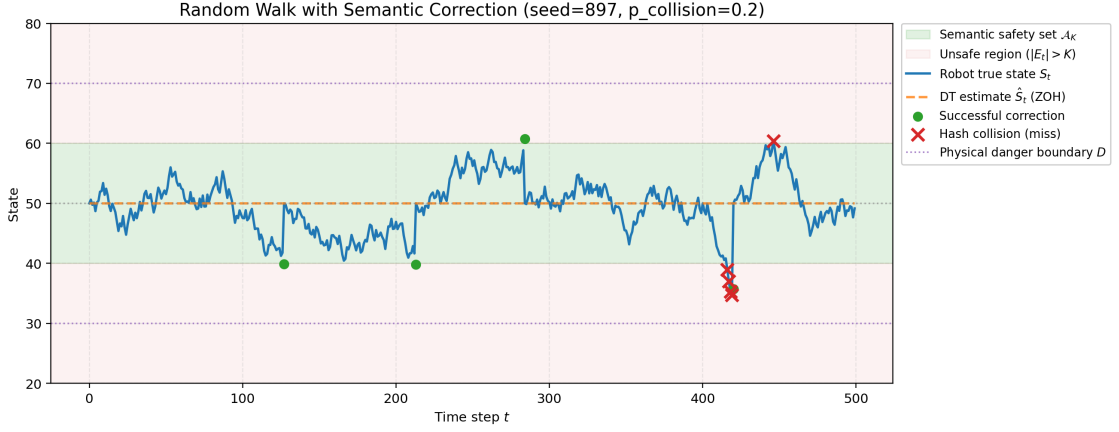


Figure 5.11: Drift-and-escape visualization for the dynamic random walk model ($K = 10$). The robot state S_t (blue) performs a random walk. The DT estimate $\hat{S}_t$ (orange, dashed) is held constant (ZOH reference). The green band indicates the semantic safety set $\mathcal{A}_K$ and the red background the unsafe region. Green dots mark successful corrections after boundary breach; red crosses mark collision-induced misses where the boundary breach is not corrected, allowing the trajectory to continue diverging ("escape").

5.4.2 Quantitative Risk (Duration and Peak Drift)

To quantify time-extended risk, we measure (i) the *breach duration* L_K (consecutive steps with $E_t > K$) and (ii) the *maximum drift* reached before recovery. Figure 5.12 reports 95th-percentile summaries over multiple runs.

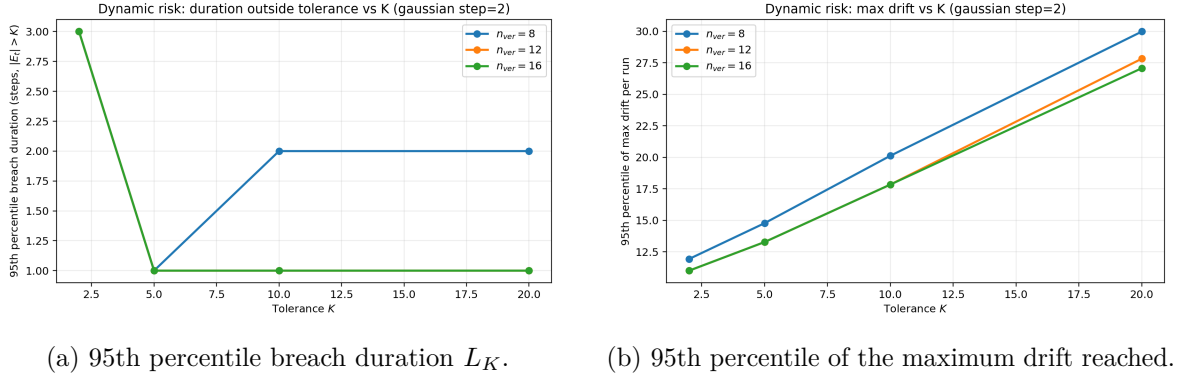


Figure 5.12: Dynamic risk vs. tolerance K (Gaussian drift). Increasing K reduces how often the boundary is checked (masking effect), but increases the miss proxy per verification event through the acceptance-set size. Short verifiers therefore yield longer boundary breaches and larger peak drift, even if the average per-tick miss probability remains small.

Masking vs. collision risk. Two distinct mechanisms drive risk as K increases. First, a larger tolerance allows the drift to grow silently for longer (masking), increasing the peak error at the moment verification begins. Second, the miss proxy grows with K (larger acceptable set), meaning that when a boundary breach does occur, it is more likely to be missed for short n_{ver} ,

extending the breach duration. This dynamic view clarifies why low verifier lengths can be dangerous even when the instantaneous collision probability seems acceptable.

Survival-style view (inter-transmission time). Beyond tail metrics of breach duration and peak drift, it is often useful for scheduling to quantify how long the system typically remains silent between transmissions. We therefore measure the inter-transmission time (IFT), defined as the number of time steps between consecutive check events ($E_t > K$). Figure 5.13 plots empirical CDFs of IFT, enabling statements such as: “with 95% probability, the system transmits at most once every X ticks”.

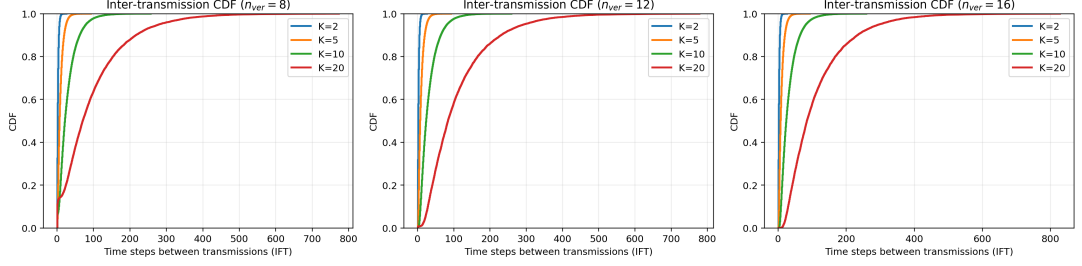


Figure 5.13: Empirical CDF of inter-transmission times (IFT) for the random-walk drift model. Larger K increases the silent intervals (masking), while smaller n_{ver} can shorten intervals by extending uncorrected excursions once a breach occurs.

6 Conclusion and Future Work

6.1 Conclusion

This thesis establishes that **ID**-based synchronization is not merely a bandwidth-saving technique, but a latency-critical enabler for edge-based **DTs**, particularly in bandwidth-constrained environments. By shifting the communication paradigm from "data reconstruction" to "state verification," we demonstrated that the **ID** protocol effectively decouples the synchronization latency from the state dimension, offering a decisive advantage over traditional transmission when the channel bandwidth drops below 1 Mbps.

Our empirical evaluation provides a definitive verdict on the engineering selection between algebraic and cryptographic encoding mechanisms for edge deployments. While **RS-ID** offers unconditional information-theoretic security, our results reveal that it suffers from severe field saturation effects at short blocklengths ($n_{ver} = 4$), rendering it impractical for low-latency control loops. In contrast, the cryptographic approach using truncated **SHA-256** maintains a stable encoding latency of approximately $2.2 \mu s$ independent of the payload size. Consequently, unless the threat model explicitly demands information-theoretic secrecy, truncated cryptographic hashing represents the superior engineering choice for **URLLC** applications, balancing computational speed with robust collision resistance.

Furthermore, the introduction of **K-ID** marks a fundamental paradigm shift from "bitwise exactness" to "semantic compliance." By relaxing the strict equality constraint to a semantic safety set $\mathcal{A}_K$, **K-ID** aligns the synchronization protocol with the control-theoretic stability margins of the physical asset. Our Pareto analysis demonstrates that introducing a moderate semantic tolerance (e.g., $K = 10$ for a state range of 100) can reduce the fallback frequency by an order of magnitude while maintaining a bounded conditional miss probability. This confirms that **K-ID** successfully trades "unnecessary data freshness" for "operational stability," validating the efficacy of Goal-Oriented Communication principles in networked control systems.

Finally, we extended the evaluation from a static (i.i.d.) state model to a dynamic drift model in which the robot evolves as a random walk and the **DT** uses a **ZOH** predictor. This time-correlated setting reveals a qualitatively different failure mode: a single collision-induced miss can persist across multiple ticks, producing extended boundary breaches and larger peak drift before recovery. As a result, risk must be characterized not only by instantaneous probabilities, but also by *duration* (how long the system remains outside tolerance) and *magnitude* (how far the drift grows before correction).

In addition, we evaluated scalability to LiDAR-scale payloads up to 5 MiB at $n_{ver} = 16$. The measured verifier generation time t_{ver} increases with payload size for both encoding mechanisms,

with [SHA-256](#) remaining faster than [RS-ID](#) in the software prototype. Importantly, at realistic bandwidths the dominant term in end-to-end latency remains the transmission time of the full payload; therefore, [ID](#) retains its primary advantage by reducing the frequency of full-payload fallbacks rather than eliminating computation.

6.2 Future Work

While this work has characterized the latency and reliability of [K-ID](#), several avenues remain to extend its theoretical depth and practical applicability in next-generation networks.

6.2.1 Energy-Aware Identification Policies

Current evaluations utilize traffic volume and latency as proxies for system efficiency. However, for battery-powered [Internet of Things \(IoT\)](#) devices, energy consumption is the ultimate constraint. Future work should explicitly model the energy–delay product, balancing the computational energy cost of hashing against the transmission energy cost of radio interfaces. By integrating the First-Order Radio Model [8] with real-time power profiling of edge hardware, we aim to derive an energy-optimal policy that dynamically adapts the tolerance K and verifier length n_{ver} based on the device’s remaining battery life—tightening safety margins when energy is abundant and relaxing them to maximize operational duration under critical battery levels.

6.2.2 Integration with 6G Physical Layer Security

The current implementation relies on synchronized [PRNGs](#), which introduces a dependency on upper-layer clock synchronization protocols like [PTP](#). To align with the architecture of 6G networks, future iterations will investigate the integration of [PLKG](#). By exploiting the channel reciprocity in [Time-Division Duplex \(TDD\)](#) systems, the robot and the [DT](#) can extract common randomness directly from the [Channel State Information \(CSI\)](#). This approach would turn channel noise into a synchronization resource, eliminating the need for clock synchronization and enhancing robustness against timing attacks in harsh industrial environments.

6.2.3 Scalability to Multi-Agent Systems

This thesis focused on a single robot–[DT](#) loop. However, industrial scenarios often involve swarms of robots sharing a physical workspace. A critical extension is the development of semantic multiple access schemes, where multiple robots superimpose their identification codes on a shared channel. Future work will explore algebraic constructions that allow the [DT](#) to verify the aggregate safety of the swarm directly in the coded domain, checking for "Semantic Inter-Robot Collisions" without decoding individual states, thereby scaling the [K-ID](#) paradigm to massive multi-agent systems.

6.2.4 Predictive Digital Twins and Adaptive K-ID

The dynamic drift evaluation in this thesis uses a deliberately simple predictor (ZOH) to isolate protocol-induced effects. A natural next step is to integrate stronger predictors such as Kalman filtering or learned kinematic models, and to study how prediction quality reshapes both the masking risk (through the breach rate) and the collision risk (through the verification rate). In addition, future work should explore adaptive policies that tune (K, n_{ver}) online based on observed breach statistics and safety budgets, tightening the tolerance under persistent drift and relaxing it during stable operation to reduce bandwidth.

6.2.5 Decoder Compute, Scheduling, and Quantization Sensitivity

Beyond bandwidth and latency, practical deployments must also account for DT-side compute and memory. Since the acceptance set size scales as $|\mathcal{A}_K| \approx 2\lfloor K/\Delta \rfloor + 1$, a direct implementation precomputes $\mathcal{V}_K$ by hashing $\mathcal{O}(|\mathcal{A}_K|)$ states per DT update, and stores up to $|\mathcal{V}_K| \leq |\mathcal{A}_K|$ verifier codewords. Future work should therefore characterize end-to-end performance under explicit decoder compute constraints (e.g., bounded hash evaluations per tick), and study algorithmic accelerations (incremental updates, caching, GPU offload).

In addition, the dynamic setting naturally suggests a scheduling-centric view: rather than only reporting per-tick risk, one can characterize the distribution of inter-transmission times (survival/CDF metrics) to answer questions like “with 99% probability, how long does the channel remain idle?” Finally, because Δ is tied to sensor precision and DT computational budget, a systematic sensitivity analysis over Δ is needed to guide robust configuration across heterogeneous sensing modalities.

Bibliography

- [1] Rudolf Ahlswede and Imre Csiszár. “Common randomness in information theory and cryptography. Part II: CR capacity”. In: *IEEE Transactions on Information Theory* 44.1 (1998), pp. 225–240.
- [2] Rudolf Ahlswede and Gunter Dueck. “Identification via Channels”. In: *1989 IEEE International Symposium on Information Theory*. 1989, pp. 15–19. DOI: [10.1109/ISIT.1989.118235](https://doi.org/10.1109/ISIT.1989.118235).
- [3] Karl J Aström and Bo M Bernhardsson. “Comparison of Riemann and Lebesgue sampling for first order stochastic systems”. In: *Proceedings of the 41st IEEE Conference on Decision and Control* 2 (2002), pp. 2011–2016.
- [4] Elaine Barker and John Kelsey. *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*. Tech. rep. NIST Special Publication 800-90A Revision 1. National Institute of Standards and Technology, 2015. DOI: [10.6028/NIST.SP.800-90Ar1](https://doi.org/10.6028/NIST.SP.800-90Ar1).
- [5] Mihir Bellare and Phillip Rogaway. “Random oracles are practical: A paradigm for designing efficient protocols”. In: *Proceedings of the 1st ACM Conference on Computer and Communications Security*. 1993, pp. 62–73.
- [6] Juan A. Cabrera et al. “6G and the Post-Shannon Theory”. In: *Shaping Future 6G Networks*. John Wiley & Sons, Ltd, 2021. Chap. 16, pp. 271–294. ISBN: 9781119765554. DOI: <https://doi.org/10.1002/9781119765554.ch16>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781119765554.ch16>.
- [7] WPMH Heemels, Karl H Johansson, and Paulo Tabuada. “An introduction to event-triggered and self-triggered control”. In: *51st IEEE Conference on Decision and Control (CDC)*. IEEE. 2012, pp. 3270–3285.
- [8] Wendi Rabiner Heinzelman, Anantha Chandrakasan, and Hari Balakrishnan. “Energy-efficient communication protocol for wireless microsensor networks”. In: *Proceedings of the 33rd Annual Hawaii International Conference on System Sciences*. IEEE. 2000, 10–pp.
- [9] Joao P Hespanha, Payam Naghshtabrizi, and Yonggang Xu. “A survey of recent results in networked control systems”. In: *Proceedings of the IEEE*. Vol. 95. 1. IEEE, 2007, pp. 138–162.
- [10] *IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*. 1588-2019. IEEE. 2019. DOI: [10.1109/IEEESTD.2020.9120376](https://doi.org/10.1109/IEEESTD.2020.9120376).

- [11] Yutaka Ishibashi and Shuji Tasaka. “Lag-compensation in delta-encoding based networked rigid body simulation”. In: *Proceedings of the 6th ACM SIGCOMM workshop on Network and system support for games*. ACM, 2007, pp. 123–128.
- [12] Sanjit Kaul, Roy Yates, and Marco Gruteser. “Real-time status: How often should one update?”. In: *2012 Proceedings IEEE INFOCOM*. IEEE. 2012, pp. 2731–2735.
- [13] Caspar von Lengerke et al. “Stopping the Data Flood: Post-Shannon Traffic Reduction in Digital-Twins Applications”. In: *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*. 2022, pp. 1–5. DOI: [10.1109/NOMS54207.2022.9789759](https://doi.org/10.1109/NOMS54207.2022.9789759).
- [14] Guyue Li et al. “Physical layer key generation in 5G and beyond wireless communications: Challenges and opportunities”. In: *Entropy* 21.5 (2019), p. 424.
- [15] Jeffrey C Mogul et al. “Potential benefits of delta encoding and data compression for HTTP”. In: *ACM SIGCOMM Computer Communication Review* 27.4 (1997), pp. 181–194.
- [16] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. Tech. rep. FIPS PUB 180-4. U.S. Department of Commerce, 2015. DOI: [10.6028/NIST.FIPS.180-4](https://doi.org/10.6028/NIST.FIPS.180-4).
- [17] Mohammad Javad Salariseddigh et al. “Deterministic K-Identification for Future Communication Networks: The Binary Symmetric Channel Results”. In: *Future Internet* 16.3 (2024). ISSN: 1999-5903. DOI: [10.3390/fi16030078](https://doi.org/10.3390/fi16030078).
- [18] Jacob T Schwartz. “Fast probabilistic algorithms for verification of polynomial identities”. In: *Journal of the ACM (JACM)* 27.4 (1980), pp. 701–717.
- [19] Emilio Calvanese Strinati et al. “Goal-Oriented and Semantic Communication in 6G AI-Native Networks: The 6G-GOALS Approach”. In: *2024 Joint European Conference on Networks and Communications and 6G Summit (EuCNC/6G Summit)*. IEEE. 2024, pp. 505–510. DOI: [10.1109/EuCNC/6GSummit60053.2024.10597087](https://doi.org/10.1109/EuCNC/6GSummit60053.2024.10597087).
- [20] Paulo Tabuada. “Event-triggered real-time scheduling of stabilizing control tasks”. In: *IEEE Transactions on Automatic Control* 52.9 (2007), pp. 1680–1685.
- [21] Ievgenii A. Tsokalo et al. “Remote Robot Control with Human-in-the-Loop over Long Distances Using Digital Twins”. In: *2019 IEEE Global Communications Conference (GLOBECOM)*. 2019, pp. 1–6. DOI: [10.1109/GLOBECOM38437.2019.9013428](https://doi.org/10.1109/GLOBECOM38437.2019.9013428).
- [22] Ping Zhang, Wenjun Xu, Hui Wang, et al. “Semantic Communication Empowered 6G Networks: Techniques, Applications, and Challenges”. In: *IEEE Access* (2025). Early Access.

A Appendix

11